OAVS TGT COMPUTER TEACHER | COMPLETE MCQ QUESTION BANK

Abstract Window Toolkit, Applets & Java Utilities

Exam-Focused MCQ Question Bank with Explanations

Syllabus Coverage: AWT Components, Graphics & Menus · Containers, Frames, Panels & FileDialog · Layout Managers (Border / Flow / Grid / Card) · Event Delegation Model & Event Categories · Event Sources, Listeners & Adapters · Anonymous Classes · Applets, Applet Life Cycle & the <applet> tag · Applet Context, AudioClip & Inter-applet Communication · Java Utility Classes (Hashtable, Vector, Math, Random, System, String, StringBuffer) · Streams, File I/O & RandomAccessFile · Reader/Writer & File Classes · Serialization & Deserialization

Total Questions: 329 | **Format:** 4-option MCQ with Answer Key & Explanation, organized topic-wise with continuous numbering (Q1–Q329)

Table of Contents

1.	AWT Components and Graphics	43 Qs
2.	Containers, Frames and Panels	19 Qs
3.	Layout Managers	19 Qs
4.	Event Delegation Model	18 Qs
5.	Event Sources, Handlers, Listeners and Adapters	17 Qs
6.	Anonymous Classes	10 Qs
7.	Applets and Applet Life Cycle	22 Qs
8.	Applet Context and Inter-applet Communication	15 Qs
9.	Java Utility Classes	64 Qs
10.	Streams and File I/O	18 Qs
11.	Reader/Writer and File Classes	28 Qs
12.	Serialization and Deserialization	17 Qs

13.	Important & Tricky Questions	18 Qs
14.	Mixed OAVS TGT Practice Set	21 Qs

1 AWT Components and Graphics

AWT BASICS

Q1. Which package in Java contains the core AWT classes?

A. java.applet
B. java.awt
C. java.util
D. java.io

Ans: B

Explanation: All Abstract Window Toolkit classes such as Component, Frame and Panel reside in the java.awt package.

AWT BASICS

Q2. Which class acts as the abstract superclass for all AWT components (Button, Label, Checkbox, etc.)?

A. Component
B. Container
C. Window
D. Object

Ans: A

Explanation: Component is the abstract base class for every visible AWT GUI element.

GRAPHICS

Q3. Which class provides methods such as drawLine(), drawOval() and drawString() for rendering on a component?

A. Toolkit
B. Graphics
C. Component
D. Canvas

Ans: B

Explanation: The abstract Graphics class encapsulates the drawing context and provides all basic drawing methods.

GRAPHICS

Q4. Which Graphics method is used to draw a filled rectangle?

A. rectFill()
B. paintRect()
C. fillRect()
D. drawRect()

Ans: C

Explanation: fillRect(x,y,width,height) draws a rectangle filled with the current color; drawRect() only draws the outline.

AWT COMPONENTS

Q5. Which AWT class is used to display non-editable single-line text on a container?

A. TextArea
B. Choice
C. TextField
D. Label

Ans: D

Explanation: Label displays a fixed, non-editable string of text and cannot receive user input directly.

AWT COMPONENTS

Q6. Which AWT class represents a clickable push button?

A. Button
B. List
C. Choice
D. Checkbox

Ans: A

Explanation: The Button class creates a labeled push button that generates an ActionEvent when clicked.

GRAPHICS

Q7. Which method is used to set the current drawing color before calling a drawing method?

A. color()
B. setPaint()
C. setForeground()
D. setColor()

Ans: D

Explanation: Graphics.setColor(Color c) sets the color used by subsequent drawing operations of that Graphics object.

AWT COMPONENTS

Q8. Which class in java.awt represents colors used for drawing and component backgrounds?

A. Palette
B. ColorModel
C. Paint
D. Color

Ans: D

Explanation: The Color class encapsulates colors in the default RGB color space, e.g. Color.RED.

AWT COMPONENTS

Q9. Which AWT component allows a user to select ONE option from a mutually exclusive group using round buttons?

A. TextField
B. Checkbox with a CheckboxGroup
C. Choice
D. List

Ans: B

Explanation: Adding several Checkbox objects to the same CheckboxGroup turns them into mutually exclusive radio buttons.

AWT COMPONENTS

Q10. Which AWT component displays a drop-down (pop-up) list from which the user selects one item?

A. TextArea
B. Scrollbar
C. Choice
D. List

Ans: D

Explanation: Choice presents a compact pop-up menu of items and shows only the currently selected one until clicked.

AWT COMPONENTS

Q11. Which AWT component provides a multi-line, editable text area?

A. Canvas
B. TextArea
C. TextField
D. Label

Ans: C

Explanation: TextArea allows multiple lines of editable text, unlike TextField which is single-line.

AWT COMPONENTS

Q12. Which AWT class is a blank rectangular area typically used for custom drawing and does not itself hold other components?

A. Frame
B. Container
C. Panel
D. Canvas

Ans: C

Explanation: Canvas is a simple, lightweight component meant purely as a surface for custom graphics.

GRAPHICS

Q13. Which Graphics method copies (moves) a rectangular area of a component to another location?

A. translateArea()
B. moveArea()
C. copyArea()
D. shiftArea()

Ans: B

Explanation: copyArea(x, y, width, height, dx, dy) copies pixels from one rectangular region to another offset location.

AWT COMPONENTS

Q14. Which AWT component lets the user select a value by dragging a slider within a range, commonly used for scrolling?

A. Slider
B. Choice
C. Scrollbar
D. List

Ans: C

Explanation: Scrollbar represents a value chosen by the user by dragging within a minimum-maximum range.

MENU COMPONENTS

Q15. Which AWT component creates a horizontal menu bar typically attached to the top of a Frame?

A. MenuBar
B. PopupMenu
C. MenuItem
D. Menu

Ans: A

Explanation: MenuBar is the container that holds one or more Menu objects and is attached to a Frame via setMenuBar().

MENU COMPONENTS

Q16. Which class represents a single drop-down menu (containing menu items) that is added to a MenuBar?

A. MenuItem
B. MenuBar
C. PopupMenu
D. Menu

Ans: D

Explanation: A Menu object holds a list of MenuItem entries and is itself added to a MenuBar, e.g. 'File' or 'Edit'.

MENU COMPONENTS

Q17. Which class represents an individual selectable entry inside a Menu?

A. CheckboxMenuItem
B. MenuBar
C. Menu
D. MenuItem

Ans: D

Explanation: MenuItem represents a single choice within a Menu, such as 'Open' or 'Save'.

MENU COMPONENTS

Q18. Which class represents a menu entry that can be toggled on/off, showing a checked boolean state?

A. MenuItem
B. CheckboxMenuItem
C. Checkbox
D. Menu

Ans: B

Explanation: CheckboxMenuItem behaves like a MenuItem but maintains and displays an on/off checked state.

MENU COMPONENTS

Q19. Which AWT class provides a context menu that pops up at an arbitrary screen position (e.g. on right-click) rather than being fixed to a MenuBar?

A. Choice
B. MenuBar
C. PopupMenu
D. Menu

Ans: C

Explanation: PopupMenu can be shown at any location on a component, typically in response to a mouse event, unlike a Menu fixed in a MenuBar.

MENU COMPONENTS

Q20. Which MenuBar method is used to attach a Menu object to it?

A. add(Menu m)
B. attach(Menu m)
C. insert(Menu m)
D. addMenu(Menu m)

Ans: A

Explanation: MenuBar.add(Menu m) appends the given Menu to the menu bar.

MENU COMPONENTS

Q21. Which Frame method associates a MenuBar with that Frame?

A. setMenu(MenuBar mb)
B. attachMenuBar(MenuBar mb)
C. setMenuBar(MenuBar mb)
D. addMenuBar(MenuBar mb)

Ans: C

Explanation: Frame.setMenuBar(MenuBar mb) attaches the given menu bar to the top of the Frame.

MENU COMPONENTS

Q22. Which listener interface detects a plain MenuItem being selected/clicked?

A. ActionListener
B. ComponentListener
C. ItemListener
D. MenuListener

Ans: A

Explanation: A regular MenuItem fires an ActionEvent on selection, handled via ActionListener's actionPerformed().

AWT COMPONENTS

Q23. Which List method adds a new item with the given text to the end of the list?

A. add(String item)
B. addItem(String item)
C. append(String item)
D. insert(String item)

Ans: A

Explanation: List.add(String item) appends a new selectable entry with the given text to the end of the list.

AWT COMPONENTS

Q24. Which List method returns the text of the currently selected item (in single-selection mode)?

A. getCurrentItem()
B. getSelectedItem()
C. getSelection()
D. getValue()

Ans: B

Explanation: getSelectedItem() returns the String of the item currently highlighted/selected in the List.

AWT COMPONENTS

Q25. Which List method allows the user to select more than one item at a time?

A. enableMultiple(true)
B. setSelectionMode(true)
C. setMultipleMode(true)
D. setMultiSelect(true)

Ans: C

Explanation: setMultipleMode(true) switches the List into a mode where multiple items can be selected simultaneously.

AWT COMPONENTS

Q26. Which pair of Checkbox methods respectively read and set its current checked (boolean) state?

A. getStatus() and setStatus(boolean b)
B. getState() and setState(boolean b)
C. isChecked() and setChecked(boolean b)
D. getValue() and setValue(boolean b)

Ans: B

Explanation: getState() returns whether the Checkbox is currently checked, and setState(boolean) programmatically changes it.

AWT COMPONENTS

Q27. Which CheckboxGroup method returns the Checkbox currently selected within that mutually exclusive group?

A. getCurrent()
B. getActiveCheckbox()
C. getSelectedCheckbox()
D. getChecked()

Ans: C

Explanation: getSelectedCheckbox() returns a reference to whichever Checkbox in the group is currently selected.

AWT COMPONENTS

Q28. Which TextField method causes typed characters to be masked (e.g. with '*'), useful for password entry?

A. hideChars(char c)
B. setPasswordChar(char c)
C. setMask(char c)
D. setEchoChar(char c)

Ans: D

Explanation: setEchoChar(char c) makes the TextField display the given character instead of the actual typed text, for password fields.

AWT COMPONENTS

Q29. Which pair of methods are used to programmatically read and update the text held in a TextField?

A. getText() and setText(String s)
B. getContent() and setContent(String s)
C. read() and write(String s)
D. getValue() and setValue(String s)

Ans: A

Explanation: getText() retrieves the current text of the field, and setText(String) replaces it.

AWT COMPONENTS

Q30. Which Label method sets how its text is horizontally positioned within the component (e.g. left, center, right)?

A. setJustify(int align)
B. setAlignment(int alignment)
C. setPosition(int pos)
D. setTextAlign(int align)

Ans: B

Explanation: setAlignment(int) accepts constants such as Label.LEFT, Label.CENTER or Label.RIGHT to position the label's text.

COMPONENT METHODS

Q31. Which Component method determines whether a component can respond to user input such as clicks or key presses?

A. setAccessible(boolean b)
B. setClickable(boolean b)
C. setActive(boolean b)
D. setEnabled(boolean b)

Ans: D

Explanation: setEnabled(false) disables a component so it appears greyed out and ignores user interaction.

COMPONENT METHODS

Q32. Which Component method controls whether a component is shown on screen at all?

A. setShown(boolean b)
B. setDisplay(boolean b)
C. show(boolean b) only
D. setVisible(boolean b)

Ans: D

Explanation: setVisible(boolean) determines whether the component is rendered and displayed to the user.

COMPONENT METHODS

Q33. Which Component method requests that keyboard input focus be moved to that component?

A. focusOn()
B. requestFocus()
C. grabFocus()
D. setFocused()

Ans: B

Explanation: requestFocus() asks the system to direct keyboard input to the calling component, if possible.

GRAPHICS

Q34. Which of these correctly describes obtaining a color in AWT?

A. Only predefined constants like Color.RED can ever be used
B. Either using a predefined constant such as Color.RED, or constructing a custom color with new Color(r, g, b)
C. The Color class cannot represent custom RGB combinations
D. Colors must always be specified as hexadecimal Strings

Ans: B

Explanation: AWT's Color class offers ready-made constants (Color.RED, Color.BLUE, etc.) and also a constructor accepting custom red, green and blue values.

AWT COMPONENTS

Q35. Which Scrollbar method sets its current value, visible amount, minimum and maximum all in a single call?

A. setRange(int min, int max)

B. setValues(int value, int visible, int min, int max)

C. setParams(int value, int visible, int min, int max)

D. configure(int value, int min, int max)

Ans: B

Explanation: setValues() lets all four key Scrollbar properties be configured together in one method call.

AWT COMPONENTS

Q36. Which pair of Scrollbar methods read and set its current scroll position value?

A. getPosition() and setPosition(int v)

B. getOffset() and setOffset(int v)

C. getValue() and setValue(int v)

D. getScroll() and setScroll(int v)

Ans: C

Explanation: getValue() returns the Scrollbar's current position, and setValue(int) programmatically updates it.

AWT COMPONENTS

Q37. Which Scrollbar method returns the size of the visible (thumb) portion relative to its scrollable range?

A. getViewSize()

B. getVisibleAmount()

C. getThumbSize()

D. getVisibleRange()

Ans: B

Explanation: getVisibleAmount() reports how much of the scrollable range is represented by the visible thumb.

AWT COMPONENTS

Q38. Which Choice method returns the index position of the currently selected item?

A. currentIndex()
B. getIndex()
C. getSelectedIndex()
D. selectedIndex()

Ans: C

Explanation: getSelectedIndex() returns the zero-based position of the item currently showing as selected.

AWT COMPONENTS

Q39. Which Choice method programmatically selects an item by its index position?

A. setIndex(int pos)
B. choose(int pos)
C. setSelected(int pos)
D. select(int pos)

Ans: D

Explanation: select(int pos) changes the currently displayed/selected item of the Choice to the one at that index.

AWT COMPONENTS

Q40. Which Choice method returns the total number of items currently present in its pop-up list?

A. count()
B. size()
C. length()
D. getItemCount()

Ans: D

Explanation: getItemCount() returns how many items have been added to the Choice component.

AWT COMPONENTS

Q41. Which TextArea method appends additional text to the end of its existing content?

A. append(String str)
B. insertAtEnd(String str)
C. concatText(String str)
D. addText(String str)

Ans: A

Explanation: append(String str) adds the given text after whatever content the TextArea already contains.

AWT COMPONENTS

Q42. Which TextArea method inserts a given string at a specified character position within its content?

A. insertAt(int pos, String str)

B. place(String str, int pos)

C. insert(String str, int pos)

D. addAt(String str, int pos)

Ans: C

Explanation: insert(str, pos) places the given text at the specified position, shifting the existing content after it.

AWT COMPONENTS

Q43. Which pair of TextArea methods configure the number of visible rows and columns it displays?

A. setLines(int rows) and setWidth(int cols)

B. setRows(int rows) and setColumns(int columns)

C. setSize(int rows, int cols)

D. setDimensions(int rows, int cols)

Ans: B

Explanation: setRows() and setColumns() independently control how many lines and how wide the TextArea appears.

2 Containers, Frames and Panels

CONTAINERS

Q44. Which class provides the capability to contain and arrange other components?

A. Component
B. Window
C. Panel
D. Container

Ans: D

Explanation: Container extends Component and adds the ability to hold and lay out child components.

FRAMES

Q45. Which class represents a top-level window having a title bar, border and optional menu bar?

A. Frame
B. Applet
C. Panel
D. Window

Ans: A

Explanation: Frame extends Window and adds a title, resizable border and menu bar support.

PANELS

Q46. Which AWT container has no title bar, cannot exist independently, and is generally used for grouping components?

A. Frame
B. Panel
C. Dialog
D. Window

Ans: B

Explanation: Panel is a concrete Container with no border or title; it must be placed inside another container such as a Frame.

CONTAINERS

Q47. Which method is used to add a component to a container?

A. append()
B. attach()
C. insert()
D. add()

Ans: D

Explanation: The Container.add(Component) method places a component inside the given container.

LAYOUT DEFAULTS

Q48. What is the default layout manager of a Frame?

A. CardLayout
B. BorderLayout
C. GridLayout
D. FlowLayout

Ans: B

Explanation: Frame (and Window/Dialog) use BorderLayout as their default layout manager.

LAYOUT DEFAULTS

Q49. What is the default layout manager of a Panel?

A. BorderLayout
B. CardLayout
C. FlowLayout
D. GridLayout

Ans: C

Explanation: Panel (and Applet) use FlowLayout as their default layout manager.

CONTAINERS

Q50. Which class is the direct superclass of Frame, providing a top-level window without borders or a menu bar?

A. Window
B. Panel
C. Dialog
D. Container

Ans: A

Explanation: Window is a top-level container with no borders/menu bar; Frame extends Window to add these features.

FRAMES

Q51. Which method sets the width and height of a Frame?

A. setSize()
B. setBounds()
C. resize()
D. setDimension()

Ans: A

Explanation: setSize(width, height) sets the Frame's dimensions; setBounds() sets both position and size together.

CONTAINERS

Q52. Which AWT class represents a top-level pop-up window, typically used for message boxes, and can be modal?

A. Frame
B. Panel
C. Dialog
D. Window

Ans: A

Explanation: Dialog is a top-level window, usually dependent on a Frame, often used for modal message or input boxes.

FRAMES

Q53. What does making a Dialog 'modal' mean?

A. It is always transparent
B. It cannot be resized
C. It has no title bar
D. It blocks input to other windows in the application until it is closed

Ans: D

Explanation: A modal Dialog blocks user interaction with other application windows until the dialog itself is dismissed.

CONTAINERS

Q54. Which method automatically resizes a Frame to fit the preferred size of its contained components?

A. autoSize()
B. resizeToFit()
C. fit()
D. pack()

Ans: C

Explanation: The pack() method sizes the window based on the preferred sizes of the components it contains.

CONTAINERS

Q55. Which class represents the blank space (top, bottom, left, right) surrounding the edges of a container?

A. Border
B. Spacing
C. Margin
D. Insets

Ans: A

Explanation: The Insets class specifies the space between a container's edge and its contained components.

CONTAINERS

Q56. Which AWT class provides automatic horizontal and/or vertical scrolling for a single child component?

A. Scrollbar
B. Panel
C. Canvas
D. ScrollPane

Ans: B

Explanation: ScrollPane is a container that automatically adds scrollbars for its single child component if needed.

CONTAINERS

Q57. Which AWT class provides a native dialog allowing the user to browse and select a file to open or save?

A. Frame
B. ScrollPane
C. Dialog
D. FileDialog

Ans: D

Explanation: FileDialog is a subclass of Dialog that displays the operating system's native file-selection dialog.

CONTAINERS

Q58. Which FileDialog mode constant configures the dialog for choosing a location to SAVE a file, rather than open one?

A. FileDialog.OUTPUT
B. FileDialog.STORE
C. FileDialog.SAVE
D. FileDialog.WRITE

Ans: C

Explanation: FileDialog.SAVE sets the dialog to save mode; FileDialog.LOAD is used for opening an existing file.

FRAMES

Q59. Which Frame method controls whether the user is allowed to resize the window by dragging its edges?

A. setFixedSize(boolean b)
B. setSizable(boolean b)
C. allowResize(boolean b)
D. setResizable(boolean b)

Ans: D

Explanation: setResizable(false) locks the Frame's size, preventing the user from dragging its borders to resize it.

FRAMES

Q60. Which Frame method sets the small icon displayed in the window's title bar and taskbar?

A. setWindowImage(Image img)
B. setTitleIcon(Image img)
C. setIconImage(Image img)
D. setIcon(Image img)

Ans: C

Explanation: setIconImage(Image img) assigns the image used to represent the Frame in its title bar and the taskbar.

CONTAINERS

Q61. Which Container method returns an array containing all the components directly added to it?

A. getChildren()
B. getComponents()
C. componentArray()
D. listComponents()

Ans: B

Explanation: getComponents() returns a Component[] array listing every component directly contained in that Container.

CONTAINERS

Q62. Which Container method removes every component that has been added to it?

A. clearAll()
B. emptyContainer()
C. deleteAll()
D. removeAll()

Ans: D

Explanation: removeAll() removes all components from the container in one call, leaving it empty.

3 Layout Managers

LAYOUT MANAGERS

Q63. Every AWT layout manager class implements which interface?

A. LayoutManager
B. Layout
C. ContainerLayout
D. Manager

Ans: A

Explanation: FlowLayout, BorderLayout, GridLayout and CardLayout all implement the LayoutManager interface.

BORDER LAYOUT

Q64. How many regions does BorderLayout divide a container into?

A. Six
B. Four
C. Three
D. Five

Ans: D

Explanation: BorderLayout provides five regions: NORTH, SOUTH, EAST, WEST and CENTER.

BORDER LAYOUT

Q65. In BorderLayout, which region occupies the remaining space after the other regions are placed?

A. CENTER
B. EAST
C. WEST
D. NORTH

Ans: A

Explanation: The CENTER region expands to fill whatever space is left after NORTH, SOUTH, EAST and WEST are allocated.

FLOW LAYOUT

Q66. What is the default alignment of components in FlowLayout?

A. RIGHT
B. LEFT
C. JUSTIFY
D. CENTER

Ans: D

Explanation: FlowLayout arranges components left to right and centers each row by default.

FLOW LAYOUT

Q67. FlowLayout arranges components in which manner?

A. In a grid of equal cells

B. One on top of another like cards

C. In fixed absolute positions

D. Left to right, wrapping to a new line when needed

Ans: D

Explanation: FlowLayout places components in a row, moving to the next line when the current row is full.

GRID LAYOUT

Q68. GridLayout arranges components in which form?

A. A rectangular grid of rows and columns of equal size

B. A single row

C. A stack of overlapping cards

D. A single column

Ans: A

Explanation: GridLayout divides the container into equal-sized rectangular cells arranged in rows and columns.

GRID LAYOUT

Q69. Which constructor correctly creates a GridLayout with 2 rows and 3 columns?

A. new GridLayout(rows=2)

B. new GridLayout(2,3)

C. new GridLayout(2)

D. new GridLayout(3,2)

Ans: B

Explanation: GridLayout(int rows, int cols) takes the number of rows first and columns second.

CARD LAYOUT

Q70. CardLayout treats each component in a container like:

A. A card in a deck, showing only one at a time

B. A tab with independent layout

C. A row in a table

D. A column in a grid

Ans: A

Explanation: CardLayout stacks components like cards; only the top (visible) card is shown at any time.

CARD LAYOUT

Q71. Which CardLayout method displays the next component in sequence?

A. show(Container)
B. move(Container)
C. forward(Container)
D. next(Container)

Ans: D

Explanation: The next(Container parent) method switches to the next card registered in that container.

CARD LAYOUT

Q72. Which method is used to display a specific named card in CardLayout?

A. display(name)
B. select(String)
C. open(name)
D. show(Container, String)

Ans: D

Explanation: show(Container parent, String name) displays the card that was added with the given name.

LAYOUT MANAGERS

Q73. Which method of Container is used to assign a layout manager to it?

A. assignLayout()
B. setLayout()
C. layout()
D. setLayoutManager()

Ans: B

Explanation: Container.setLayout(LayoutManager mgr) sets the layout manager used to arrange its components.

LAYOUT MANAGERS

Q74. If setLayout(null) is used on a container, what happens?

A. The container throws a runtime exception
B. No components can be added
C. A default FlowLayout is applied automatically
D. Components must be positioned manually using setBounds()

Ans: D

Explanation: Passing null disables automatic layout, requiring absolute positioning via setBounds() on each component.

FLOW LAYOUT

Q75. In addition to CENTER, which other alignments can be explicitly specified for FlowLayout?

A. No other alignment is possible
B. NORTH and SOUTH
C. Only TOP and BOTTOM
D. LEFT and RIGHT

Ans: C

Explanation: FlowLayout's constructor accepts FlowLayout.LEFT, FlowLayout.CENTER or FlowLayout.RIGHT as alignment constants.

FLOW LAYOUT

Q76. What do the 'hgap' and 'vgap' parameters control in a FlowLayout constructor?

A. The horizontal and vertical gap (spacing) between components
B. The border thickness
C. The number of rows and columns
D. The height and width of the container

Ans: A

Explanation: hgap and vgap set the horizontal and vertical spacing left between neighbouring components and container edges.

GRID LAYOUT

Q77. If a GridLayout is constructed with the number of rows set to 0, what does that indicate?

A. The number of rows is unspecified and is calculated based on columns and component count
B. Zero components are allowed
C. The layout becomes a FlowLayout
D. An exception is always thrown

Ans: B

Explanation: Specifying 0 for rows (or columns) tells GridLayout to compute that dimension automatically from the other value.

BORDER LAYOUT

Q78. When adding a component to a BorderLayout container without specifying a region, where is it placed by default?

A. NORTH
B. It is not added
C. CENTER
D. WEST

Ans: A

Explanation: If no region constant is given, add() places the component in the CENTER region by default.

BORDER LAYOUT

Q79. Which String constant is equivalent to BorderLayout.CENTER when using the string-based add() form?

A. "Middle"
B. "Center"
C. "Main"
D. "C"

Ans: B

Explanation: BorderLayout also accepts region names as Strings such as "Center", "North", "South", "East", "West".

CARD LAYOUT

Q80. Which CardLayout method immediately switches display to the very first card that was added?

A. home(Container)
B. start(Container)
C. first(Container)
D. top(Container)

Ans: C

Explanation: first(Container parent) shows the first component added to that container under CardLayout.

CARD LAYOUT

Q81. Which CardLayout method switches the display to the last card that was added?

A. bottom(Container)
B. end(Container)
C. last(Container)
D. final(Container)

Ans: B

Explanation: last(Container parent) displays the last card that was added to the container.

4 Event Delegation Model

EVENT MODEL

Q82. In the Event Delegation Model, the object that generates an event is called the:

A. Adapter
B. Handler
C. Listener
D. Source

Ans: D

Explanation: The component on which the event occurs (e.g. a Button being clicked) is called the event source.

EVENT MODEL

Q83. In the Event Delegation Model, which object is responsible for receiving and processing an event?

A. Source
B. Container
C. Listener
D. Delegate

Ans: C

Explanation: A listener is an object that implements a listener interface and is registered to handle events from a source.

EVENT MODEL

Q84. A source registers interest in receiving event notifications by calling a method of the form:

A. addXxxListener()
B. registerEvent()
C. bindListener()
D. attachHandler()

Ans: A

Explanation: Sources provide addXxxListener() methods, e.g. addActionListener(), to register listener objects.

EVENT MODEL

Q85. All AWT event classes are ultimately subclasses of which class?

A. java.util.EventObject
B. java.io.Serializable
C. java.awt.Component
D. java.lang.Object

Ans: A

Explanation: AWTEvent, and hence ActionEvent, MouseEvent etc., trace back to java.util.EventObject.

EVENT MODEL

Q86. Which package contains most AWT event classes and listener interfaces?

A. javax.event
B. java.awt.event
C. java.util.event
D. java.event

Ans: B

Explanation: Classes like ActionEvent, MouseEvent and interfaces like ActionListener are defined in java.awt.event.

EVENT MODEL

Q87. Which method of an event object returns the object on which the event originally occurred?

A. getComponent()
B. getSource()
C. getEvent()
D. getOrigin()

Ans: B

Explanation: EventObject.getSource() returns a reference to the object that generated the event.

EVENT MODEL

Q88. The Event Delegation Model is preferred over the older event-inheritance model mainly because it:

A. Requires no interfaces
B. Cannot handle multiple listeners
C. Works only with applets
D. Delegates event handling to separate listener objects, improving efficiency

Ans: D

Explanation: It separates event generation from handling and allows one or more listener objects to process events efficiently.

EVENT MODEL

Q89. Can a single event source have more than one listener registered on it?

A. Yes, multiple listeners can be registered
B. Only for MouseEvent
C. No, only one listener is allowed
D. Only in applets

Ans: A

Explanation: Multiple listener objects can be registered on the same source; all registered listeners are notified.

EVENT MODEL

Q90. Which ActionEvent method returns the command string associated with the event (e.g. a button's label)?

A. getText()
B. getLabel()
C. getActionCommand()
D. getCommand()

Ans: D

Explanation: getActionCommand() returns the command name associated with the action, typically the button/menu item label.

EVENT MODEL

Q91. Which MouseEvent methods return the x and y coordinates of the mouse pointer at the time of the event?

A. getPosX() and getPosY()
B. locationX() and locationY()
C. getX() and getY()
D. xPos() and yPos()

Ans: C

Explanation: MouseEvent.getX() and getY() return the horizontal and vertical coordinates relative to the source component.

EVENT MODEL

Q92. Which KeyEvent method returns an integer code identifying which key was pressed?

A. getKeyCode()
B. getKey()
C. getKeyChar()
D. getCode()

Ans: B

Explanation: getKeyCode() returns a virtual key code identifying the physical key, e.g. KeyEvent.VK_ENTER.

EVENT MODEL

Q93. Which event class and listener pair is used to detect when a component is added to or removed from a container?

A. ContainerEvent / ContainerListener
B. WindowEvent / WindowListener
C. ComponentEvent / ComponentListener
D. ItemEvent / ItemListener

Ans: C

Explanation: ContainerEvent, handled by ContainerListener, is fired when components are added to or removed from a container.

EVENT MODEL

Q94. Which event class and listener pair detects a component being resized, moved, shown or hidden?

A. ActionEvent / ActionListener
B. FocusEvent / FocusListener
C. AdjustmentEvent / AdjustmentListener
D. ComponentEvent / ComponentListener

Ans: D

Explanation: ComponentListener handles ComponentEvent notifications such as componentResized() and componentMoved().

EVENT MODEL

Q95. Which superclass do all low-level AWT event classes (like MouseEvent, KeyEvent) extend, which itself extends java.util.EventObject?

A. InputEvent
B. AWTEvent
C. ComponentEvent
D. ActionEvent

Ans: A

Explanation: AWTEvent is the root class for all AWT events; low-level events like KeyEvent and MouseEvent extend it (via InputEvent).

EVENT MODEL

Q96. Which method would you call to stop a previously registered ActionListener from receiving further events from a source?

A. deleteActionListener(ActionListener l)
B. detachActionListener(ActionListener l)
C. removeActionListener(ActionListener l)
D. unregisterListener(ActionListener l)

Ans: C

Explanation: Every addXxxListener() method has a matching removeXxxListener() method used to deregister a listener from a source.

EVENT CATEGORIES

Q97. AWT events are broadly classified into semantic events and low-level events. Which best describes this distinction?

A. Semantic events are always faster to process than low-level events

B. Semantic events represent a meaningful user action on a component (e.g. a button being activated); low-level events represent a raw physical input or component-state change (e.g. a key press or window move)

C. There is no real distinction -- the terms are interchangeable

D. Semantic events only occur in applets, low-level events only occur in applications

Ans: B

Explanation: Semantic events (ActionEvent, ItemEvent, AdjustmentEvent, TextEvent) convey what the user accomplished, while low-level events (KeyEvent, MouseEvent, FocusEvent, WindowEvent, ComponentEvent) convey raw input or state changes.

EVENT CATEGORIES

Q98. Which of the following is classified as a SEMANTIC event rather than a low-level event?

A. MouseEvent

B. KeyEvent

C. WindowEvent

D. ActionEvent

Ans: D

Explanation: ActionEvent represents a higher-level, meaningful action (like a button click), making it a semantic event; the others are low-level events.

EVENT CATEGORIES

Q99. Which of the following is classified as a LOW-LEVEL event rather than a semantic event?

A. AdjustmentEvent

B. ItemEvent

C. MouseEvent

D. ActionEvent

Ans: C

Explanation: MouseEvent reports raw pointer input (press, release, movement) and is a low-level event; the others listed are semantic events.

5 Event Sources, Handlers, Listeners and Adapters

LISTENERS

Q100. Which method must a class implement when it implements the ActionListener interface?

A. onAction()
B. actionOccurred()
C. actionPerformed()
D. performAction()

Ans: C

Explanation: ActionListener declares a single method, actionPerformed(ActionEvent e), invoked when the action event fires.

LISTENERS

Q101. How many methods does the MouseListener interface declare?

A. Five
B. Two
C. Four
D. Three

Ans: A

Explanation: MouseListener declares five methods: mouseClicked, mousePressed, mouseReleased, mouseEntered and mouseExited.

LISTENERS

Q102. Which interface is used to detect key press and release events?

A. FocusListener
B. ActionListener
C. KeyListener
D. MouseListener

Ans: C

Explanation: KeyListener provides keyPressed(), keyReleased() and keyTyped() methods for keyboard events.

LISTENERS

Q103. Which listener interface's single method itemStateChanged() is used with Checkbox and Choice components?

A. ItemListener
B. AdjustmentListener
C. TextListener
D. ActionListener

Ans: A

Explanation: ItemListener handles state-change events from components like Checkbox, Choice and List.

ADAPTERS

Q104. What is the main purpose of an adapter class such as MouseAdapter or WindowAdapter?

A. To register multiple sources at once
B. To provide default empty implementations so a subclass overrides only the methods it needs
C. To generate events automatically
D. To replace the need for listener interfaces entirely

Ans: B

Explanation: Adapter classes implement all methods of a multi-method listener interface with empty bodies, so a subclass can override only the relevant ones.

ADAPTERS

Q105. Which adapter class would you extend to conveniently handle only the windowClosing() event?

A. ComponentAdapter
B. CloseAdapter
C. WindowAdapter
D. FrameAdapter

Ans: C

Explanation: WindowAdapter implements all seven WindowListener methods; overriding just windowClosing() is enough.

LISTENERS

Q106. How many methods does the WindowListener interface declare in total?

A. Eight
B. Seven
C. Six
D. Five

Ans: B

Explanation: WindowListener declares seven methods, including windowOpened, windowClosing and windowActivated.

LISTENERS

Q107. Which listener interface is used to detect when a component gains or loses input focus?

A. KeyListener
B. ItemListener
C. FocusListener
D. ComponentListener

Ans: C

Explanation: FocusListener declares focusGained() and focusLost() methods.

ADAPTERS

Q108. Which of these is NOT a valid AWT adapter class?

A. WindowAdapter
B. KeyAdapter
C. ActionAdapter
D. MouseAdapter

Ans: C

Explanation: There is no ActionAdapter because ActionListener has only one method, so an adapter is unnecessary.

LISTENERS

Q109. Which MouseMotionListener method is invoked when the mouse is moved with a button held down?

A. mouseSlide()
B. mouseTracked()
C. mouseDragged()
D. mouseMoved()

Ans: C

Explanation: mouseDragged() fires while the mouse moves with a button pressed; mouseMoved() fires with no button pressed.

LISTENERS

Q110. Which listener interface's single method adjustmentValueChanged() responds to Scrollbar movement?

A. SliderListener
B. AdjustmentListener
C. ScrollListener
D. ChangeListener

Ans: C

Explanation: AdjustmentListener is used with Scrollbar (and ScrollPane) to detect changes in the scroll position.

LISTENERS

Q111. Which listener interface responds to changes in the text of a TextField or TextArea as the user types?

A. TextListener
B. KeyListener
C. DocumentListener
D. InputListener

Ans: A

Explanation: TextListener's textValueChanged() method fires whenever the text content of a text component changes.

ADAPTERS

Q112. Which adapter class provides empty default methods for the ComponentListener interface?

A. ComponentHandler
B. ComponentEvent
C. ComponentBase
D. ComponentAdapter

Ans: D

Explanation: ComponentAdapter implements ComponentListener with empty method bodies for convenient partial overriding.

ADAPTERS

Q113. Which adapter class provides empty default methods for the ContainerListener interface?

A. ContainerAdapter
B. ContainerBase
C. ContainerHandler
D. PanelAdapter

Ans: C

Explanation: ContainerAdapter implements ContainerListener with empty bodies so only needed methods must be overridden.

LISTENERS

Q114. Which listener interface's single method mouseWheelMoved() detects scroll-wheel activity?

A. WheelListener
B. ScrollListener
C. MouseListener
D. MouseWheelListener

Ans: C

Explanation: MouseWheelListener provides mouseWheelMoved(MouseWheelEvent e) to respond to mouse wheel rotation.

LISTENERS

Q115. A class that implements a listener interface with five methods but only needs one of them MUST still:

A. Mark the class abstract
B. Provide implementations (even if empty) for all five methods, or extend the corresponding adapter instead
C. Leave the other four unimplemented
D. Override only the one method it needs

Ans: B

Explanation: Implementing an interface directly requires providing every declared method; using the adapter class avoids this by allowing selective overriding.

LISTENERS

Q116. Which WindowListener method is invoked specifically when a window is minimized to an icon?

A. windowIconified(WindowEvent e)
B. windowMinimized(WindowEvent e)
C. windowClosed(WindowEvent e)
D. windowDeactivated(WindowEvent e)

Ans: A

Explanation: windowIconified() fires when the user minimizes the window; windowDeiconified() fires when it is restored.

6 Anonymous Classes

ANONYMOUS CLASSES

Q117. An anonymous class in Java is best described as:

A. A class defined and instantiated in a single expression without a name

B. A class declared without any methods

C. A class that must be declared static

D. A class that can never be instantiated

Ans: A

Explanation: An anonymous class combines class definition and object creation in one expression and has no name.

ANONYMOUS CLASSES

Q118. Anonymous classes are most commonly used in AWT programs for:

A. Implementing event listeners inline

B. Defining Frame subclasses only

C. Creating Color objects

D. Declaring utility constants

Ans: A

Explanation: They provide a quick, compact way to implement a listener interface directly at the point of registration.

ANONYMOUS CLASSES

Q119. Can an anonymous class have an explicit, named constructor?

A. Only if it extends Object

B. Only if it implements an interface

C. No, since it has no class name to define a constructor with

D. Yes, always

Ans: C

Explanation: Since an anonymous class has no name, it cannot declare a constructor of its own.

ANONYMOUS CLASSES

Q120. An anonymous class can do which of the following?

A. Extend multiple classes simultaneously

B. Neither extend nor implement anything

C. Implement multiple interfaces simultaneously

D. Extend one class OR implement one interface at a time

Ans: D

Explanation: An anonymous class may extend a single class or implement a single interface, not both together.

ANONYMOUS CLASSES

Q121. Which statement about anonymous classes is TRUE?

A. They generate a class file named after the outer class with a numeric suffix

B. They are declared using the 'anon' keyword

C. They must be declared public

D. They cannot access final local variables of the enclosing method

Ans: A

Explanation: The compiler generates a class file like Outer$1.class for each anonymous class defined inside Outer.

ANONYMOUS CLASSES

Q122. Why are anonymous classes convenient for handling simple, one-off events like a button click?

A. They automatically remove the listener after use

B. They avoid creating a separate named top-level listener class for a single use

C. They execute faster than named classes

D. They are the only way to implement ActionListener

Ans: B

Explanation: They let a listener be written inline where it's used, avoiding an extra separately-named class for one-time use.

ANONYMOUS CLASSES

Q123. What is the correct general syntax pattern for creating an anonymous class that implements an interface?

A. Interface anon = new() { };
B. new ClassName() extends Interface { }
C. class Interface() { };
D. new Interface() { // method overrides };

Ans: B

Explanation: An anonymous implementing class is written as 'new InterfaceName() { ...method bodies... };' directly at the point of use.

ANONYMOUS CLASSES

Q124. Compared to a separately declared named inner class implementing a listener, an anonymous class listener mainly trades away:

A. The ability to catch the event at all
B. Reusability, since it cannot be instantiated again elsewhere in the code
C. Access to the enclosing class's members
D. The ability to override methods

Ans: A

Explanation: Because it has no name, an anonymous class's definition cannot be reused to create further instances elsewhere.

ANONYMOUS CLASSES

Q125. An anonymous class defined inside a method can access local variables of that method only if those variables are:

A. static
B. transient
C. declared public
D. effectively final (not reassigned after initialization)

Ans: A

Explanation: Anonymous (and inner) classes may only capture local variables that are final or effectively final.

ANONYMOUS CLASSES

Q126. Anonymous classes are a special case of which broader Java feature?

A. Abstract classes only

B. Packages

C. Inner (nested) classes

D. Interfaces

Ans: D

Explanation: An anonymous class is essentially an unnamed, one-off inner class defined and instantiated together.

7 Applets and Applet Life Cycle

APPLETS

Q127. The Applet class belongs to which package?

A. javax.applet
B. java.applet
C. java.util
D. java.awt

Ans: B

Explanation: The Applet class is defined in the java.applet package.

APPLETS

Q128. The Applet class is a subclass of which AWT class?

A. Frame
B. Window
C. Container
D. Panel

Ans: D

Explanation: Applet extends Panel, so it inherits FlowLayout as its default layout and can hold components.

APPLET LIFE CYCLE

Q129. Which applet life-cycle method is called only once, when the applet is first loaded?

A. init()
B. paint()
C. start()
D. destroy()

Ans: A

Explanation: init() performs one-time initialization and is called exactly once during an applet's life.

APPLET LIFE CYCLE

Q130. Which method is called every time the applet becomes active, including after init()?

A. paint()
B. stop()
C. destroy()
D. start()

Ans: D

Explanation: start() is called immediately after init() and again each time the applet is revisited/becomes visible.

APPLET LIFE CYCLE

Q131. Which method is responsible for rendering the applet's output on screen?

A. init()
B. start()
C. stop()
D. paint()

Ans: D

Explanation: paint(Graphics g) is called to draw the applet's visible content and may be invoked repeatedly.

APPLET LIFE CYCLE

Q132. Which method is called when the browser leaves the page containing the applet (applet becomes inactive)?

A. init()
B. destroy()
C. stop()
D. pause()

Ans: C

Explanation: stop() suspends applet activity when the user navigates away, and can be restarted via start().

APPLET LIFE CYCLE

Q133. Which method is called once, just before the applet is removed from memory?

A. finalize()
B. init()
C. destroy()
D. stop()

Ans: C

Explanation: destroy() performs final cleanup and is called exactly once at the end of an applet's life.

APPLET LIFE CYCLE

Q134. What is the correct order of the first-time applet life-cycle method calls?

A. init() -> paint() -> start()
B. init() -> start() -> paint()
C. start() -> init() -> paint()
D. paint() -> init() -> start()

Ans: B

Explanation: On loading, init() runs first for setup, then start() begins execution, then paint() renders output.

APPLETS

Q135. Why do applets run under a restrictive 'sandbox' security model?

A. Because Panel requires it

B. To improve rendering speed

C. Because they are downloaded from a network and must be prevented from harming the local system

D. To allow unrestricted file access

Ans: C

Explanation: Since applet code is downloaded and executed in a browser, sandbox restrictions limit file and network access to prevent misuse.

APPLETS

Q136. Which method is used inside an applet to retrieve a value specified by a <param> tag in the HTML page?

A. readParam(String)

B. getParameter(String)

C. getAttribute(String)

D. fetchParam(String)

Ans: B

Explanation: getParameter(String name) reads the value of a named parameter passed from the HTML <param> tag.

APPLET LIFE CYCLE

Q137. Which method is automatically called by the system whenever paint() needs to be re-invoked (e.g. after clearing the screen), and by default it calls paint()?

A. refresh()

B. reload()

C. repaint()

D. update()

Ans: C

Explanation: update(Graphics g) is called before paint() on a repaint request; by default it clears the background then calls paint().

APPLET LIFE CYCLE

Q138. Which method should a program call to REQUEST that an applet's display be redrawn, rather than calling paint() directly?

A. paint()
B. display()
C. repaint()
D. redraw()

Ans: A

Explanation: repaint() schedules a call to update()/paint() via the AWT event thread rather than drawing immediately.

APPLETS

Q139. Which method returns the URL of the directory containing the HTML page that invoked the applet?

A. getCodeBase()
B. getDocumentBase()
C. getHomeBase()
D. getURL()

Ans: A

Explanation: getDocumentBase() returns the URL of the document (HTML page) in which the applet is embedded.

APPLETS

Q140. Which method returns the URL of the directory containing the applet's class file itself?

A. getDocumentBase()
B. getClassURL()
C. getCodeBase()
D. getSourceBase()

Ans: A

Explanation: getCodeBase() returns the URL of the directory from which the applet's .class file was loaded.

APPLETS

Q141. Which method allows an applet to report descriptive information about itself, such as author and version?

A. describe()
B. getAppletInfo()
C. aboutApplet()
D. getInfo()

Ans: D

Explanation: Overriding getAppletInfo() lets an applet return a String describing itself, its author and version.

APPLETS

Q142. Which boolean method tells an applet whether it is currently active (running), typically true between start() and stop()?

A. isRunning()
B. isAlive()
C. isStarted()
D. isActive()

Ans: D

Explanation: isActive() returns true once start() has been called and before stop() is called.

APPLETS

Q143. Which of the following restrictions typically applies to an untrusted applet running in a browser sandbox?

A. It has unrestricted access to all system resources
B. It can read and write any file on the client's local disk
C. It cannot ordinarily read or write files on the local client system
D. It can freely connect to any remote host on the internet

Ans: B

Explanation: Sandboxed applets are generally barred from local file access and can usually connect only back to their own originating host.

APPLETS

Q144. Which attribute of the HTML <applet> tag specifies the name of the compiled .class file to run?

A. code
B. src
C. file
D. class

Ans: A

Explanation: The 'code' attribute names the applet's compiled class file, e.g. <applet code="MyApplet.class" ...>.

APPLETS

Q145. Which two attributes of the <applet> tag specify the display dimensions of the applet's rectangular area on the page?

A. width and height
B. rows and cols
C. size and scale
D. w and h

Ans: A

Explanation: The width and height attributes (in pixels) define how much space the browser reserves for the applet.

APPLETS

Q146. Which statement correctly distinguishes an applet from a standalone Java application?

A. An applet always runs faster than an application
B. An application cannot access the local file system while an applet can
C. An applet has no main() method and runs inside a browser or appletviewer under security restrictions, while an application runs independently via main()
D. There is essentially no real difference between the two

Ans: C

Explanation: Applets are embedded, sandboxed programs started by the browser through their life-cycle methods, unlike applications which start execution at main().

APPLETS

Q147. Which <applet> tag attribute specifies the directory to load the applet's class file from, if different from the HTML page's own location?

A. base
B. source
C. codepath
D. codebase

Ans: D

Explanation: The 'codebase' attribute optionally overrides the default location (the HTML page's own directory) used to locate the applet's class file.

APPLETS

Q148. Which method can an applet override to describe the names, types and meaning of the parameters it supports via <param> tags?

A. paramInfo()

B. getParameters()

C. getParameterInfo()

D. describeParams()

Ans: C

Explanation: getParameterInfo() returns a String[][] array documenting each supported parameter's name, type and description.

8 Applet Context and Inter-applet Communication

APPLET CONTEXT

Q149. The AppletContext interface represents:

A. The environment (browser) in which the applet is running

B. The applet's graphics buffer

C. A list of applet parameters

D. The applet's own thread

Ans: A

Explanation: AppletContext lets an applet obtain information about, and communicate with, its browsing environment.

APPLET CONTEXT

Q150. Which method is used to obtain an applet's AppletContext object?

A. context()

B. getBrowserContext()

C. getAppletContext()

D. getContext()

Ans: C

Explanation: Applet.getAppletContext() returns the AppletContext associated with the current applet.

INTER-APPLET COMMUNICATION

Q151. To communicate with another named applet on the same HTML page, which AppletContext method is used?

A. getPeer(String name)

B. locateApplet(String name)

C. findApplet(String name)

D. getApplet(String name)

Ans: D

Explanation: getApplet(String name) returns a reference to another applet on the page having the given name, enabling inter-applet communication.

INTER-APPLET COMMUNICATION

Q152. Which method returns an Enumeration of all applets on the current page?

A. getApplets()
B. listApplets()
C. applets()
D. getAllApplets()

Ans: A

Explanation: AppletContext.getApplets() returns an Enumeration of every applet present on the same page.

APPLET CONTEXT

Q153. Which AppletContext method is used to display a message in the browser's status line?

A. displayStatus(String)
B. printStatus(String)
C. setStatus(String)
D. showStatus(String)

Ans: D

Explanation: showStatus(String msg) requests the browser to display the given message in its status bar.

APPLET CONTEXT

Q154. Which AppletContext method can be used to make the browser display a new web document/URL?

A. navigate(URL)
B. loadPage(URL)
C. openDocument(URL)
D. showDocument(URL)

Ans: D

Explanation: showDocument(URL url) instructs the browser to replace the currently displayed page with the given URL.

APPLET CONTEXT

Q155. Which AppletContext method retrieves an audio clip located at a given URL so the applet can play it?

A. getAudioClip(URL)
B. getSound(URL)
C. fetchAudio(URL)
D. loadClip(URL)

Ans: A

Explanation: getAudioClip(URL url) returns an AudioClip object that the applet can play using its play() method.

APPLET CONTEXT

Q156. Which AppletContext method retrieves an Image object from a given URL for use in the applet?

A. fetchImage(URL)
B. loadImage(URL)
C. getPicture(URL)
D. getImage(URL)

Ans: D

Explanation: getImage(URL url) returns an Image object that the applet can subsequently draw using Graphics.drawImage().

INTER-APPLET COMMUNICATION

Q157. For getApplet(String name) to successfully retrieve another applet, what condition must be true?

A. Both applets must be running on the same page and the target applet must have a matching NAME attribute
B. The requesting applet must be inactive
C. The applets must run in separate browser windows
D. Both applets must extend the same custom subclass

Ans: A

Explanation: getApplet() only finds applets present on the same HTML page that were given a NAME attribute matching the requested string.

INTER-APPLET COMMUNICATION

Q158. Since getApplets() returns an Enumeration, which method is used to step through its elements one at a time?

A. getNext()
B. hasMoreElements() and nextElement()
C. next()
D. iterator()

Ans: C

Explanation: Enumeration provides hasMoreElements() to check for more items and nextElement() to retrieve each one in turn.

APPLET CONTEXT

Q159. Which interface represents a sound clip that an applet can play, typically obtained via getAudioClip()?

A. Clip
B. Media
C. AudioClip
D. Sound

Ans: C

Explanation: AudioClip (java.applet.AudioClip) represents an audio clip with play(), loop() and stop() methods.

APPLET CONTEXT

Q160. Which AudioClip method plays the sound once, from start to finish?

A. run()
B. play()
C. execute()
D. start()

Ans: B

Explanation: play() plays the audio clip a single time from the beginning.

APPLET CONTEXT

Q161. Which AudioClip method plays the sound continuously, repeating until explicitly stopped?

A. cycle()
B. repeat()
C. replay()
D. loop()

Ans: D

Explanation: loop() causes the clip to play repeatedly until stop() is called.

APPLET CONTEXT

Q162. Which AudioClip method halts playback of a clip that is currently playing or looping?

A. stop()
B. end()
C. pause()
D. halt()

Ans: A

Explanation: stop() immediately stops the AudioClip, whether it was playing once or looping continuously.

APPLET CONTEXT

Q163. Which overloaded AppletContext method directs a specific named browser frame/window to display a given URL?

A. openDocument(URL url, String target)

B. loadDocument(URL url, String target)

C. showDocument(URL url, String target)

D. showDocument(URL url, int frame)

Ans: C

Explanation: The two-argument showDocument(URL, String target) form lets the applet specify a target frame (e.g. "_blank") for the new page.

9 Java Utility Classes

HASHTABLE

Q164. The Hashtable class stores data in the form of:

A. Sorted character sequences
B. Only unique values without keys
C. Ordered index-value pairs
D. Key-value pairs

Ans: D

Explanation: Hashtable, in java.util, maps unique keys to values, similar to a dictionary.

HASHTABLE

Q165. Which of the following is TRUE about the Hashtable class?

A. It does not allow null keys or null values
B. It allows a null key but not a null value
C. It is unsynchronized
D. It allows null keys and null values

Ans: A

Explanation: Unlike HashMap, Hashtable does not permit a null key or a null value and is synchronized.

VECTOR

Q166. The Vector class implements a:

A. Singly linked list only
B. Fixed-size array
C. Key-value map
D. Growable (dynamic) array of objects

Ans: D

Explanation: Vector is a legacy class in java.util representing a dynamic array that grows as needed.

VECTOR

Q167. Which Vector method adds an element to the end of the vector?

A. insertElement()
B. addElement()
C. append()
D. push()

Ans: B

Explanation: addElement(Object obj) appends the specified element to the end of the Vector.

VECTOR

Q168. Is the Vector class synchronized (thread-safe) by default?

A. Yes
B. No
C. Only when explicitly declared synchronized
D. Only in single-threaded programs

Ans: A

Explanation: Vector's methods are synchronized by default, making it thread-safe, unlike ArrayList.

MATH CLASS

Q169. Which of the following statements about the Math class is TRUE?

A. All its methods are static
B. It belongs to java.util
C. It only works with integers
D. It must be instantiated with 'new' before use

Ans: A

Explanation: Math is a final class in java.lang whose methods (pow, sqrt, abs, etc.) are all static.

MATH CLASS

Q170. Which Math method returns the value of x raised to the power y?

A. Math.power(x,y)
B. Math.raise(x,y)
C. Math.pow(x,y)
D. Math.exp(x,y)

Ans: C

Explanation: Math.pow(double x, double y) computes x raised to the power y.

MATH CLASS

Q171. What range of values does Math.random() return?

A. 0.0 to 1.0 inclusive of both ends
B. 0.0 (inclusive) to 1.0 (exclusive)
C. 1 to 100
D. -1.0 to 1.0

Ans: B

Explanation: Math.random() returns a double value greater than or equal to 0.0 and strictly less than 1.0.

RANDOM CLASS

Q172. The Random class, used to generate pseudo-random numbers, belongs to which package?

A. java.io
B. java.math
C. java.lang
D. java.util

Ans: D

Explanation: java.util.Random provides methods like nextInt() and nextDouble() to generate pseudo-random values.

RANDOM CLASS

Q173. What is the purpose of supplying a seed value to a Random object?

A. To produce a reproducible sequence of pseudo-random numbers
B. To restrict output to positive numbers only
C. To increase performance only
D. To make the numbers truly random

Ans: A

Explanation: A given seed always produces the same sequence of pseudo-random numbers, useful for repeatable testing.

SYSTEM CLASS

Q174. Which System method is used to terminate the currently running Java program?

A. System.halt()
B. System.terminate()
C. System.stop()
D. System.exit(int status)

Ans: D

Explanation: System.exit(int status) immediately terminates the JVM; a non-zero status typically indicates abnormal termination.

SYSTEM CLASS

Q175. Which System method returns the current time in milliseconds since the epoch?

A. System.currentTimeMillis()
B. System.now()
C. System.getTime()
D. System.time()

Ans: A

Explanation: System.currentTimeMillis() returns the current time as a long value, often used to measure elapsed time.

STRING CLASS

Q176. Which statement about the String class is correct?

A. String objects are mutable

B. String extends StringBuffer

C. String objects are immutable; any modification creates a new object

D. Strings can be resized in place

Ans: C

Explanation: Once created, a String object's contents cannot change; operations like concatenation return a new String.

STRINGBUFFER

Q177. How does StringBuffer differ from String?

A. StringBuffer is immutable like String

B. StringBuffer represents a mutable sequence of characters that can be modified in place

C. StringBuffer belongs to java.util

D. StringBuffer cannot be modified after creation

Ans: B

Explanation: StringBuffer allows in-place modification via methods like append(), insert() and reverse(), unlike immutable String.

HASHTABLE

Q178. Which Hashtable method is used to insert a key-value pair into the table?

A. add(key, value)

B. set(key, value)

C. insert(key, value)

D. put(key, value)

Ans: A

Explanation: put(Object key, Object value) associates the specified value with the specified key in the Hashtable.

HASHTABLE

Q179. Which Hashtable method checks whether a specified key is already present in the table?

A. isKey()

B. containsKey()

C. hasKey()

D. keyExists()

Ans: C

Explanation: containsKey(Object key) returns true if the Hashtable contains a mapping for the specified key.

VECTOR

Q180. Which Vector method removes the first occurrence of a specified element from the vector?

A. removeElement(Object obj)
B. deleteElement()
C. discard(Object obj)
D. clear(Object obj)

Ans: B

Explanation: removeElement(Object obj) removes the first (lowest-indexed) occurrence of obj from the Vector.

VECTOR

Q181. What is the difference between a Vector's 'size' and its 'capacity'?

A. Size refers to memory in bytes
B. Capacity is always smaller than size
C. Size is the number of elements actually stored; capacity is the total storage currently allocated
D. They always mean the same thing

Ans: D

Explanation: Size counts the current elements, while capacity is the allocated internal array length, which grows as needed.

MATH CLASS

Q182. Which Math method rounds a floating-point number up to the smallest integer value not less than the argument?

A. Math.floor()
B. Math.ceil()
C. Math.round()
D. Math.trunc()

Ans: A

Explanation: Math.ceil(double a) returns the smallest double value greater than or equal to a, i.e. rounds upward.

MATH CLASS

Q183. Which Math method rounds a floating-point number down to the largest integer value not greater than the argument?

A. Math.int()
B. Math.ceil()
C. Math.round()
D. Math.floor()

Ans: B

Explanation: Math.floor(double a) returns the largest double value less than or equal to a, i.e. rounds downward.

STRING CLASS

Q184. Which String method returns the position of the first occurrence of a specified character or substring?

A. find()
B. search()
C. locate()
D. indexOf()

Ans: A

Explanation: indexOf() returns the index of the first occurrence of the given character/substring, or -1 if not found.

STRINGBUFFER

Q185. Which StringBuffer method removes the characters in a specified substring range?

A. erase(int start, int end)
B. delete(int start, int end)
C. remove()
D. cut(int start, int end)

Ans: C

Explanation: delete(int start, int end) removes the characters from index start up to (but not including) end.

HASHTABLE

Q186. Which Hashtable method removes the key-value mapping for a specified key?

A. discard(Object key)
B. remove(Object key)
C. clear(Object key)
D. delete(Object key)

Ans: B

Explanation: remove(Object key) deletes the entry for the given key from the Hashtable, if present.

HASHTABLE

Q187. Which Hashtable method returns an Enumeration of all the keys stored in the table?

A. keySet()
B. listKeys()
C. keys()
D. getKeys()

Ans: C

Explanation: keys() returns an Enumeration over all keys currently in the Hashtable.

HASHTABLE

Q188. Which Hashtable method returns an Enumeration of all the values stored in the table?

A. elements()
B. listValues()
C. getElements()
D. values()

Ans: A

Explanation: elements() returns an Enumeration over all the values currently stored in the Hashtable.

VECTOR

Q189. Which Vector method returns true if the vector currently contains no elements?

A. isEmpty()
B. size() == null
C. isBlank()
D. isNull()

Ans: A

Explanation: isEmpty() returns true when the Vector's size is zero.

VECTOR

Q190. Which Vector method directly returns the first element stored in the vector?

A. getFirst()
B. firstElement()
C. top()
D. head()

Ans: B

Explanation: firstElement() is the Vector convenience method returning the element at index 0.

VECTOR

Q191. Which Vector method checks whether a specified object is present anywhere within the vector?

A. hasElement(Object elem)
B. contains(Object elem)
C. exists(Object elem)
D. includes(Object elem)

Ans: B

Explanation: contains(Object elem) returns true if the Vector holds at least one element equal to elem.

MATH CLASS

Q192. Which Math method returns the absolute (non-negative) value of its numeric argument?

A. Math.unsign()
B. Math.magnitude()
C. Math.abs()
D. Math.positive()

Ans: C

Explanation: Math.abs(x) returns x if x is positive, and -x if x is negative, giving a non-negative result.

MATH CLASS

Q193. Which Math method returns the larger of two given values?

A. Math.top()
B. Math.greater()
C. Math.max()
D. Math.highest()

Ans: C

Explanation: Math.max(a, b) returns whichever of a or b is greater.

MATH CLASS

Q194. Which Math method returns the positive square root of its argument?

A. Math.radical()
B. Math.root()
C. Math.power(x, 0.5)
D. Math.sqrt()

Ans: D

Explanation: Math.sqrt(x) computes and returns the positive square root of x.

MATH CLASS

Q195. Which Math field provides the constant value of pi (approximately 3.14159)?

A. Math.CONST_PI
B. Math.PIE
C. Math.PI
D. Math.pi

Ans: C

Explanation: Math.PI is a public static final double field holding the value of the mathematical constant pi.

SYSTEM CLASS

Q196. Which System method efficiently copies a range of elements from one array into another?

A. System.duplicate()
B. System.arraycopy()
C. System.transfer()
D. System.copyArray()

Ans: B

Explanation: System.arraycopy(src, srcPos, dest, destPos, length) performs a fast, native array copy.

SYSTEM CLASS

Q197. Which System method retrieves the value of a named system property, such as "java.version"?

A. System.getConfig(String key)
B. System.getProperty(String key)
C. System.readProperty(String key)
D. System.getEnv(String key)

Ans: B

Explanation: System.getProperty(String key) returns the value of the specified JVM/system property.

STRING CLASS

Q198. Which is the correct way to compare the CONTENTS of two String objects for equality?

A. Using the == operator (which compares references)

B. Using the compareTo() method's result as true/false

C. Using the hashCode() values directly

D. Using the equals() method

Ans: D

Explanation: equals() compares the actual character sequences, while == merely compares whether both references point to the same object.

STRING CLASS

Q199. Which String method converts the string into an array of individual characters?

A. splitChars()

B. charArray()

C. getChars() with no arguments

D. toCharArray()

Ans: D

Explanation: toCharArray() returns a new char[] containing each character of the String in order.

STRING CLASS

Q200. Which String method returns a copy of the string with leading and trailing whitespace removed?

A. trim()

B. compact()

C. truncate()

D. clean()

Ans: A

Explanation: trim() removes whitespace from both ends of the String and returns the resulting copy.

STRINGBUFFER

Q201. Which StringBuffer method reverses the order of characters currently in the buffer?

A. flip()

B. swap()

C. invert()

D. reverse()

Ans: D

Explanation: reverse() replaces the character sequence in the StringBuffer with its mirror-image reverse.

VECTOR

Q202. Which Vector method inserts a specified element at a given index, shifting later elements to the right?

A. putElementAt(Object obj, int index)
B. insertElementAt(Object obj, int index)
C. addElementAt(Object obj, int index)
D. setAt(Object obj, int index)

Ans: B

Explanation: insertElementAt(obj, index) places obj at the given index, shifting subsequent elements one position later.

VECTOR

Q203. Which Vector method removes every element from the vector, leaving it empty but still usable?

A. deleteAllElements()
B. clearAll()
C. emptyVector()
D. removeAllElements()

Ans: D

Explanation: removeAllElements() empties the Vector of all its elements, resetting its size to zero.

VECTOR

Q204. Which Vector method replaces the element at a specified index with a new value?

A. setElementAt(Object obj, int index)
B. updateElementAt(Object obj, int index)
C. replaceElementAt(Object obj, int index)
D. putAt(Object obj, int index)

Ans: A

Explanation: setElementAt(obj, index) overwrites the existing element at that index with the new value obj.

HASHTABLE

Q205. Which pair of Hashtable methods respectively report the number of entries stored and remove every entry?

A. length() and reset()
B. size() and clear()
C. total() and wipe()
D. count() and empty()

Ans: B

Explanation: size() returns the number of key-value mappings, while clear() removes all mappings from the Hashtable.

HASHTABLE

Q206. What is the key difference between Hashtable's contains(Object value) and containsKey(Object key) methods?

A. contains() checks if a given VALUE exists anywhere in the table, while containsKey() checks if a given KEY exists

B. Both only check keys, never values

C. They are completely identical in behavior

D. contains() checks keys, containsKey() checks values

Ans: A

Explanation: This is a classic trick: the legacy contains() method actually tests values, not keys -- containsKey() is the one that tests keys.

STRING CLASS

Q207. Why can two separately created String literals with identical content sometimes be found equal using the == operator?

A. Because the JVM ignores reference comparison for Strings

B. Because == always compares String content, not references

C. Because String literals are mutable and merge automatically

D. Because identical String literals are typically stored once in a shared string pool, so both references point to the same object

Ans: D

Explanation: Java maintains a string pool for literals; identical literal values usually share the same object reference, unlike Strings created with 'new'.

STRINGBUFFER

Q208. What is the difference between a StringBuffer's length() and capacity()?

A. capacity() returns characters used; length() returns total allocated space

B. length() returns the number of characters currently stored; capacity() returns the total allocated storage before the buffer must grow

C. They always return the same value

D. length() refers to byte size, capacity() to character count

Ans: B

Explanation: length() is how many characters are actually present, while capacity() is the current size of the underlying buffer, which is usually larger.

HASHTABLE

Q209. Which Hashtable method retrieves the value associated with a specified key?

A. get(Object key)
B. find(Object key)
C. fetch(Object key)
D. value(Object key)

Ans: A

Explanation: get(Object key) returns the value mapped to the given key, or null if the key is not present.

MATH CLASS

Q210. Which Math method returns the smaller of two given values?

A. Math.lowest()
B. Math.bottom()
C. Math.lesser()
D. Math.min()

Ans: D

Explanation: Math.min(a, b) returns whichever of a or b is the smaller value.

MATH CLASS

Q211. Which Math method rounds a floating-point value to the nearest whole number (long/int)?

A. Math.nearest()
B. Math.integer()
C. Math.whole()
D. Math.round()

Ans: D

Explanation: Math.round(x) rounds x to the closest integer value, rounding half-way cases upward.

VECTOR

Q212. Which Vector method directly returns the last element stored in the vector?

A. tail()
B. endElement()
C. lastElement()
D. getLast()

Ans: C

Explanation: lastElement() is the Vector convenience method returning the element at the highest index.

STRING CLASS

Q213. Which String method returns the number of characters in the string?

A. getLength()
B. count()
C. length()
D. size()

Ans: C

Explanation: length() returns the total number of characters contained in the String.

STRING CLASS

Q214. Which String method returns the character located at a specified index?

A. getChar(int index)
B. indexChar(int index)
C. charAt(int index)
D. characterAt(int index)

Ans: C

Explanation: charAt(int index) returns the char value at the given position within the String.

STRING CLASS

Q215. Which String method extracts and returns a portion of the string between given indices?

A. extract(int beginIndex, int endIndex)
B. substring(int beginIndex, int endIndex)
C. subString(int from, int to)
D. slice(int beginIndex, int endIndex)

Ans: B

Explanation: substring(begin, end) returns the characters from beginIndex up to, but not including, endIndex.

STRING CLASS

Q216. Which String method joins (appends) another string to the end of the current string, returning a new String?

A. concat(String str)
B. join(String str)
C. append(String str)
D. merge(String str)

Ans: A

Explanation: concat(String str) returns a new String formed by appending str to the end of the original string.

STRING CLASS

Q217. Which String method compares two strings lexicographically and returns a negative, zero, or positive int?

A. equals(String anotherString)
B. compareTo(String anotherString)
C. matches(String anotherString)
D. sortWith(String anotherString)

Ans: B

Explanation: compareTo() returns an int indicating whether the string is lexicographically less than, equal to, or greater than the argument.

STRING CLASS

Q218. Which String method returns a new string with all occurrences of a given character replaced by another character?

A. change(char oldChar, char newChar)
B. substitute(char oldChar, char newChar)
C. replace(char oldChar, char newChar)
D. swap(char oldChar, char newChar)

Ans: C

Explanation: replace(oldChar, newChar) returns a new String with every occurrence of oldChar replaced by newChar.

STRING CLASS

Q219. Which pair of String methods convert a string to all-uppercase and all-lowercase respectively?

A. toUpper() and toLower()
B. asUpper() and asLower()
C. toUpperCase() and toLowerCase()
D. upperCase() and lowerCase()

Ans: C

Explanation: toUpperCase() and toLowerCase() return new String copies converted to the respective letter case.

STRING CLASS

Q220. Which String method splits a string into an array of substrings based on a given delimiter?

A. divide(String regex)
B. tokenize(String regex)
C. partition(String regex)
D. split(String regex)

Ans: D

Explanation: split(String regex) breaks the string around matches of the given delimiter and returns a String[] array.

STRING CLASS

Q221. Which static String method converts a value of another type (such as int or boolean) into its String representation?

A. String.valueOf()
B. String.convert()
C. String.toString()
D. String.parse()

Ans: A

Explanation: String.valueOf(x) is a static method that returns the String representation of the given argument.

STRINGBUFFER

Q222. Which StringBuffer method inserts a given string at a specified index within the buffer, shifting existing characters?

A. add(int offset, String str)
B. push(int offset, String str)
C. insert(int offset, String str)
D. place(int offset, String str)

Ans: C

Explanation: insert(offset, str) inserts str into the buffer starting at the given offset, shifting later characters forward.

STRINGBUFFER

Q223. Which StringBuffer method returns the character at a specified index within the buffer?

A. valueAt(int index)
B. characterAt(int index)
C. charAt(int index)
D. getChar(int index)

Ans: C

Explanation: charAt(int index) returns the char currently stored at the given position in the StringBuffer.

STRINGBUFFER

Q224. Which StringBuffer method replaces the character at a specified index with a new character?

A. setCharAt(int index, char ch)
B. replaceCharAt(int index, char ch)
C. updateCharAt(int index, char ch)
D. putCharAt(int index, char ch)

Ans: A

Explanation: setCharAt(index, ch) overwrites the character at the given position with ch, modifying the buffer in place.

STRINGBUFFER

Q225. Which StringBuffer method removes the single character at a specified index?

A. removeCharAt(int index)
B. clearCharAt(int index)
C. dropCharAt(int index)
D. deleteCharAt(int index)

Ans: D

Explanation: deleteCharAt(index) removes the character at the given index, shifting subsequent characters left by one.

RANDOM CLASS

Q226. Besides nextInt(), which other Random methods generate a pseudo-random double value and a pseudo-random boolean value respectively?

A. getDouble() and getBoolean()
B. nextDouble() and nextBoolean()
C. nextFloat() and nextFlag()
D. randomDouble() and randomBoolean()

Ans: B

Explanation: Random provides nextDouble() (0.0 to 1.0) and nextBoolean() (true/false) alongside nextInt() for different data types.

SYSTEM CLASS

Q227. Which three standard streams are predefined as public static fields of the System class?

A. System.print, System.read and System.error
B. System.output, System.input and System.fault
C. System.console, System.stream and System.log
D. System.out, System.in and System.err

Ans: D

Explanation: System.out and System.err are standard output/error PrintStreams, while System.in is the standard input InputStream.

10 Streams and File I/O

STREAMS BASICS

Q228. Java's I/O classes for reading and writing data are primarily found in which package?

A. java.lang
B. java.io
C. java.net
D. java.util

Ans: B

Explanation: The java.io package provides the classes and interfaces for input/output operations, including streams.

STREAMS BASICS

Q229. Java streams are broadly classified into byte streams and:

A. Character streams
B. Buffer streams only
C. Object streams only
D. Bit streams

Ans: A

Explanation: Byte streams handle raw 8-bit data while character streams handle 16-bit Unicode character data.

BYTE STREAMS

Q230. Which abstract class is the superclass of all byte input stream classes?

A. OutputStream
B. Writer
C. Reader
D. InputStream

Ans: D

Explanation: InputStream is the abstract base class for byte-oriented input streams such as FileInputStream.

BYTE STREAMS

Q231. Which abstract class is the superclass of all byte output stream classes?

A. Writer
B. Reader
C. InputStream
D. OutputStream

Ans: D

Explanation: OutputStream is the abstract base class for byte-oriented output streams such as FileOutputStream.

BYTE STREAMS

Q232. Which class is used to read raw bytes directly from a file?

A. FileReader
B. FileInputStream
C. BufferedReader
D. FileWriter

Ans: B

Explanation: FileInputStream reads raw byte data from a file, extending InputStream.

BYTE STREAMS

Q233. Which class is used to write raw bytes directly to a file?

A. PrintWriter
B. FileWriter
C. BufferedWriter
D. FileOutputStream

Ans: D

Explanation: FileOutputStream writes raw byte data to a file, extending OutputStream.

STREAM CLASSES

Q234. Which InputStream method returns the number of bytes that can be read without blocking?

A. remaining()
B. available()
C. size()
D. length()

Ans: B

Explanation: available() returns an estimate of the number of bytes that can be read without blocking.

STREAM CLASSES

Q235. Which class wraps another stream to add buffering and improve I/O performance for bytes?

A. BufferedInputStream
B. FileInputStream
C. ObjectInputStream
D. DataInputStream

Ans: A

Explanation: BufferedInputStream adds an internal buffer around another InputStream to reduce the number of physical reads.

STREAM CLASSES

Q236. Which stream classes are used to read and write Java's primitive data types (int, double, etc.) directly?

A. DataInputStream / DataOutputStream
B. FileInputStream / FileOutputStream
C. BufferedReader / BufferedWriter
D. ObjectInputStream / ObjectOutputStream

Ans: A

Explanation: DataInputStream and DataOutputStream provide methods like readInt()/writeInt() for primitive data types.

STREAM CLASSES

Q237. Which OutputStream method must often be called to ensure buffered data is actually written out?

A. write()
B. close()
C. sync()
D. flush()

Ans: D

Explanation: flush() forces any buffered output bytes to be written to their intended destination immediately.

STREAM CLASSES

Q238. Which class wraps an OutputStream to add buffering, improving performance by reducing the number of physical write operations?

A. DataOutputStream
B. BufferedOutputStream
C. FileOutputStream
D. PrintStream

Ans: B

Explanation: BufferedOutputStream stores bytes in an internal buffer and writes them out in larger chunks for efficiency.

STREAM CLASSES

Q239. Which class provides convenient methods like println() for writing formatted text and is used internally by System.out?

A. DataOutputStream
B. BufferedWriter
C. PrintStream
D. ObjectOutputStream

Ans: C

Explanation: PrintStream adds functionality to write representations of various data values conveniently, e.g. println().

STREAMS BASICS

Q240. Which InputStream method actually reads and returns the next single byte of data, or -1 at end of stream?

A. get()
B. read()
C. fetch()
D. next()

Ans: A

Explanation: read() with no arguments reads one byte and returns it as an int (0-255), or -1 if the end of the stream is reached.

STREAMS BASICS

Q241. Which OutputStream method is used to write a single byte of data to the stream?

A. put(int b)
B. push(int b)
C. write(int b)
D. send(int b)

Ans: A

Explanation: write(int b) writes the specified byte to the output stream.

STREAM CLASSES

Q242. Which DataInputStream method reads and returns the next value from the stream specifically as an int?

A. readInt()
B. nextInt()
C. readInteger()
D. getInt()

Ans: C

Explanation: DataInputStream.readInt() reads four bytes and reconstructs them as a Java int value.

RANDOMACCESSFILE

Q243. Which class allows a program to both read from and write to a file at arbitrary byte positions, rather than only sequentially?

A. RandomAccessFile
B. SequentialFile
C. FileInputStream
D. BufferedRandomAccess

Ans: A

Explanation: RandomAccessFile supports non-sequential (random) access, letting a program jump to and read/write at any position in a file.

RANDOMACCESSFILE

Q244. Which RandomAccessFile method moves its internal file pointer to a specified byte offset within the file?

A. moveTo(long pos)
B. seek(long pos)
C. jump(long pos)
D. setPosition(long pos)

Ans: B

Explanation: seek(long pos) repositions the file pointer so the next read/write operation occurs starting at that byte offset.

RANDOMACCESSFILE

Q245. Which pair of interfaces does RandomAccessFile implement, giving it access to methods for reading and writing Java's primitive data types?

A. Serializable and Externalizable
B. InputStream and OutputStream
C. DataInput and DataOutput
D. Readable and Writable

Ans: C

Explanation: By implementing both DataInput and DataOutput, RandomAccessFile supports methods like readInt(), writeInt(), readUTF() and writeUTF().

11 Reader/Writer and File Classes

READER/WRITER

Q246. Which abstract class is the superclass of all character input stream classes?

A. Writer
B. InputStream
C. Reader
D. OutputStream

Ans: C

Explanation: Reader is the abstract base class for character-oriented input, handling 16-bit Unicode characters.

READER/WRITER

Q247. Which abstract class is the superclass of all character output stream classes?

A. Reader
B. InputStream
C. Writer
D. OutputStream

Ans: C

Explanation: Writer is the abstract base class for character-oriented output streams.

READER/WRITER

Q248. Which class is used to read character data directly from a file?

A. FileWriter
B. BufferedInputStream
C. FileInputStream
D. FileReader

Ans: D

Explanation: FileReader is a convenience class for reading character data from files, extending Reader.

READER/WRITER

Q249. Which class is used to write character data directly to a file?

A. FileOutputStream
B. FileWriter
C. PrintStream
D. FileReader

Ans: B

Explanation: FileWriter is a convenience class for writing character data to files, extending Writer.

READER/WRITER

Q250. Which BufferedReader method reads an entire line of text as a String?

A. nextLine()
B. readLine()
C. readText()
D. getLine()

Ans: B

Explanation: BufferedReader.readLine() reads a line of text and returns it as a String, or null at end of stream.

READER/WRITER

Q251. Which class acts as a bridge, converting a byte stream into a character stream?

A. OutputStreamWriter
B. BufferedReader
C. InputStreamReader
D. DataInputStream

Ans: C

Explanation: InputStreamReader wraps a byte InputStream and decodes bytes into characters using a specified charset.

FILE CLASS

Q252. The File class in java.io primarily represents:

A. A stream of bytes
B. The actual content of a file
C. A serialized object
D. An abstract representation of a file or directory pathname

Ans: D

Explanation: File does not read or write data; it represents path/metadata information about a file or directory.

FILE CLASS

Q253. Which File class method checks whether a given pathname actually exists on disk?

A. isFile()
B. canRead()
C. exists()
D. isPresent()

Ans: C

Explanation: exists() returns true if the file or directory denoted by the File object is found on the file system.

FILE CLASS

Q254. Which File class method is used to create a new directory?

A. mkdir()
B. createDirectory()
C. newFolder()
D. makeDir()

Ans: A

Explanation: File.mkdir() attempts to create the directory named by this File's pathname.

FILE CLASS

Q255. Which File class method permanently removes the file or empty directory it represents?

A. erase()
B. delete()
C. remove()
D. clear()

Ans: B

Explanation: File.delete() deletes the file or directory denoted by this abstract pathname.

FILE CLASS

Q256. Which File class method returns an array of Strings naming the files and directories within a directory?

A. list()
B. children()
C. getContents()
D. listFiles()

Ans: D

Explanation: File.list() returns a String array of the names of files and directories inside the directory it represents.

FILE CLASS

Q257. Which File class method returns an array of File objects (not just names) for the entries in a directory?

A. children()
B. getFiles()
C. listFiles()
D. list()

Ans: D

Explanation: listFiles() returns an array of File objects representing each entry in the directory, unlike list() which returns Strings.

FILE CLASS

Q258. Which File class method renames or moves the file represented by the File object?

A. renameTo(File dest)
B. move(File dest)
C. relocate(File dest)
D. rename(String name)

Ans: B

Explanation: renameTo(File dest) attempts to rename (or move) the abstract file to the given destination pathname.

FILE CLASS

Q259. Which File class method returns the time the file was last modified?

A. getTimestamp()
B. modifiedTime()
C. lastModified()
D. getModifiedDate()

Ans: D

Explanation: lastModified() returns a long representing the time the file was last modified, in milliseconds since the epoch.

READER/WRITER

Q260. Which Writer class allows characters to be buffered before being written to an underlying Writer, improving efficiency?

A. OutputStreamWriter
B. BufferedWriter
C. FileWriter
D. PrintWriter

Ans: C

Explanation: BufferedWriter wraps another Writer and buffers characters to minimize the number of costly write operations.

FILE CLASS

Q261. Which File method returns the absolute pathname string of the file or directory it represents?

A. getFullPath()
B. getAbsolutePath()
C. getCanonicalPath() only
D. absolutePath()

Ans: B

Explanation: getAbsolutePath() returns the complete absolute path corresponding to this File object.

FILE CLASS

Q262. Which File method returns the pathname of the file's parent directory, or null if it has none?

A. getParent()
B. getRoot()
C. parentDir()
D. getParentPath()

Ans: A

Explanation: getParent() returns the parent directory's path as a String, or null if the File has no parent.

FILE CLASS

Q263. Which File method tests whether the file or directory is marked as hidden by the underlying file system?

A. isSystem()
B. isInvisible()
C. isHidden()
D. isConcealed()

Ans: C

Explanation: isHidden() returns true if the file is flagged as hidden according to the host operating system's convention.

FILE CLASS

Q264. Which File method returns the length (size in bytes) of the file it represents?

A. size()
B. fileLength()
C. length()
D. getSize()

Ans: C

Explanation: length() returns the file's size in bytes as a long value.

FILE CLASS

Q265. Which File field provides the platform-dependent character used to separate directory names within a pathname?

A. File.divider
B. File.pathChar
C. File.delimiter
D. File.separator

Ans: D

Explanation: File.separator is a String holding the system-dependent path separator, e.g. "/" on Unix or "\\" on Windows.

FILE CLASS

Q266. Which File method creates the directory named by this File object, along with any necessary but nonexistent PARENT directories?

A. mkdirAll()
B. mkdirs()
C. createDirs()
D. mkdir()

Ans: B

Explanation: mkdirs() creates the full directory path including missing parent directories, unlike mkdir() which creates only one level and fails if the parent is missing.

FILE CLASS

Q267. Which File method creates a new, empty file at the given pathname only if a file with that name does not already exist?

A. createNewFile()
B. touch()
C. newFile()
D. makeFile()

Ans: A

Explanation: createNewFile() atomically creates a new empty file and returns false (rather than overwriting) if the file already exists.

FILE CLASS

Q268. Which File method tests whether the pathname represented is an absolute path rather than a relative one?

A. isAbsolute()
B. isComplete()
C. isFull()
D. isRooted()

Ans: A

Explanation: isAbsolute() returns true if the File's pathname is already a complete, absolute path.

READER/WRITER

Q269. Which class acts as a bridge, converting a character stream into a byte OutputStream for writing (the writer-side counterpart to InputStreamReader)?

A. InputStreamReader
B. OutputStreamWriter
C. BufferedWriter
D. PrintWriter

Ans: B

Explanation: OutputStreamWriter encodes characters into bytes using a specified charset and writes them to an underlying OutputStream.

FILE CLASS

Q270. Which File method tests whether the pathname represented is a directory rather than a regular file?

A. isFolder()
B. isDirectory()
C. isPath()
D. isDir()

Ans: B

Explanation: isDirectory() returns true if the File object denotes an existing directory.

FILE CLASS

Q271. Which pair of File methods test whether the application can read from and write to the given file respectively?

A. canRead() and canWrite()
B. readable() and writable()
C. isReadable() and isWritable()
D. hasReadAccess() and hasWriteAccess()

Ans: A

Explanation: canRead() and canWrite() check the file's read and write permissions for the running application.

READER/WRITER

Q272. Which character-stream class provides convenient formatted output methods such as println(), serving as the Writer-based counterpart to PrintStream?

A. FileWriter
B. PrintWriter
C. PrintStream
D. BufferedWriter

Ans: B

Explanation: PrintWriter offers the same convenient print()/println() style output as PrintStream, but works with character Writers instead of byte streams.

READER/WRITER

Q273. In a PrintWriter constructor such as PrintWriter(Writer out, boolean autoFlush), what does setting the boolean parameter to true cause?

A. The output buffer is automatically flushed after every println() call (and on certain other operations)

B. Every character is converted to uppercase before writing

C. The stream automatically closes itself after each write

D. Writing switches from character mode to byte mode

Ans: A

Explanation: The autoFlush parameter, when true, ensures buffered output is pushed out promptly after line-terminating writes, rather than staying buffered indefinitely.

12 Serialization and Deserialization

SERIALIZATION

Q274. What is serialization in Java?

A. Encrypting a Java file

B. Compiling a Java class into bytecode

C. Converting an object's state into a byte stream for storage or transmission

D. Sorting an array of objects

Ans: C

Explanation: Serialization converts an object's state into a byte stream so it can be saved to a file or sent over a network.

SERIALIZATION

Q275. Which interface must a class implement to make its objects serializable?

A. Serializable

B. Streamable

C. Cloneable

D. Externalizable

Ans: A

Explanation: Serializable is a marker interface (no methods) that signals the JVM that objects of the class may be serialized.

SERIALIZATION

Q276. Which class's writeObject() method is used to serialize an object to a stream?

A. DataOutputStream

B. FileOutputStream

C. ObjectOutputStream

D. ObjectInputStream

Ans: C

Explanation: ObjectOutputStream.writeObject(obj) serializes the given object and writes its byte representation to the stream.

DESERIALIZATION

Q277. Which class's readObject() method is used to reconstruct an object from a serialized stream?

A. ObjectInputStream
B. DataInputStream
C. ObjectOutputStream
D. FileInputStream

Ans: A

Explanation: ObjectInputStream.readObject() reads bytes from the stream and reconstructs the original object.

SERIALIZATION

Q278. Which keyword is used to prevent a particular field of a class from being serialized?

A. static
B. final
C. transient
D. volatile

Ans: C

Explanation: Fields marked transient are skipped during serialization and are not part of the saved byte stream.

SERIALIZATION

Q279. Why is a serialVersionUID declared in a serializable class?

A. To speed up object creation
B. To encrypt the serialized data
C. To make the class abstract
D. To uniquely identify the class version for compatibility during deserialization

Ans: D

Explanation: serialVersionUID verifies that the sender and receiver of a serialized object have compatible class versions.

SERIALIZATION

Q280. Are static fields of a class included when an object of that class is serialized?

A. Only primitive static fields

B. No, static fields belong to the class, not the object, and are not serialized

C. Only if marked transient

D. Yes, always

Ans: B

Explanation: Since static fields belong to the class rather than any instance, they are not saved during serialization.

SERIALIZATION

Q281. How does the Externalizable interface differ from Serializable?

A. Externalizable is a marker interface with no methods, same as Serializable

B. Externalizable cannot be used for file storage

C. Externalizable gives the programmer full control of the serialization process via writeExternal()/readExternal()

D. Externalizable does not require implementing any interface

Ans: C

Explanation: Unlike the marker interface Serializable, Externalizable requires implementing writeExternal() and readExternal() for custom serialization logic.

SERIALIZATION

Q282. If a class implements Serializable and has a superclass that does NOT implement Serializable, what happens to the superclass's fields during deserialization?

A. They are serialized normally like any other field

B. A compile-time error occurs

C. The whole object fails to deserialize

D. They are initialized using the superclass's no-argument constructor, not restored from the stream

Ans: A

Explanation: Non-serializable superclass state is not saved; upon deserialization its fields are set by invoking its no-arg constructor.

SERIALIZATION

Q283. Can an array of objects be serialized in Java?

A. Only arrays of primitive types can be serialized

B. Yes, provided every object in the array is itself Serializable (or null)

C. No, arrays can never be serialized

D. Only single-dimensional arrays of Strings

Ans: C

Explanation: Arrays are serializable as long as their component type/elements are serializable; ObjectOutputStream handles them directly.

DESERIALIZATION

Q284. What happens if the serialVersionUID of a class does not match the UID stored in a serialized object being read back?

A. The object is automatically upgraded

B. An InvalidClassException is typically thrown

C. Deserialization proceeds normally and silently ignores the mismatch

D. Nothing; UIDs are cosmetic only

Ans: C

Explanation: A serialVersionUID mismatch signals an incompatible class change and normally causes an InvalidClassException during deserialization.

SERIALIZATION

Q285. Which of these best describes why the Externalizable interface can be more efficient than plain Serializable for some classes?

A. It skips writing any data at all

B. The programmer controls exactly what is written/read, avoiding the overhead of default reflection-based serialization

C. It requires no interface implementation

D. It compresses data automatically

Ans: A

Explanation: Because writeExternal()/readExternal() are written explicitly by the programmer, unnecessary reflection-based overhead of default serialization can be avoided.

SERIALIZATION

Q286. Which method, if implemented in Externalizable, is responsible for restoring an object's state while deserializing?

A. deserialize()
B. readExternal()
C. restoreState()
D. readObject()

Ans: D

Explanation: Externalizable requires readExternal(ObjectInput in) to explicitly read back and restore the object's fields.

SERIALIZATION

Q287. Which Externalizable method is responsible for WRITING an object's state out during serialization, the counterpart to readExternal()?

A. writeExternal(ObjectOutput out)
B. saveExternal(ObjectOutput out)
C. writeObject(ObjectOutput out)
D. serializeExternal(ObjectOutput out)

Ans: A

Explanation: writeExternal(ObjectOutput out) is where the programmer explicitly writes whatever state needs to be preserved.

DESERIALIZATION

Q288. If a new, optional field is simply added to a class (same serialVersionUID kept) and an OLD serialized object of that class is deserialized using the NEW class definition, what typically happens?

A. Deserialization always fails with an exception
B. The old object's existing data is silently discarded entirely
C. Deserialization succeeds; the new field is set to its default value (e.g. null or 0) since it wasn't present in the old stream
D. The JVM automatically recompiles the old serialized data

Ans: C

Explanation: Adding fields is generally a backward-compatible change; fields absent from the old stream simply take their default values.

SERIALIZATION

Q289. Which statement about a class implementing Externalizable is TRUE?

A. transient fields are forcibly included automatically

B. The programmer must explicitly write every field that needs to be preserved, since nothing is saved automatically

C. It behaves identically to default Serializable behavior

D. All fields are saved automatically regardless of what is written

Ans: B

Explanation: Unlike default Serializable behavior, Externalizable saves nothing unless the programmer explicitly writes it in writeExternal().

SERIALIZATION

Q290. If a Serializable object contains a reference to another object as a field, what happens to that referenced object when the outer object is serialized?

A. It is silently ignored and left out of the serialized data

B. Only its class name is saved, not its data

C. It causes a compile-time error unless marked transient

D. The referenced object is automatically serialized too, provided it is also Serializable, preserving the full object graph

Ans: D

Explanation: Java's serialization automatically follows and serializes the entire graph of referenced objects, as long as each one is itself Serializable.

13 Important & Tricky Questions

ASSERTION-REASON

Q291. Assertion (A): Panel is commonly used as an invisible container to group related AWT components.
Reason (R): Panel has no title bar or border and cannot be displayed independently of another container.
Choose the correct option:

A. Both A and R are true, and R is the correct explanation of A

B. A is false but R is true

C. A is true but R is false

D. Both A and R are true, but R is NOT the correct explanation of A

Ans: A

Explanation: Panel is borderless and must sit inside another container, which is exactly why it is ideal for grouping components invisibly.

ASSERTION-REASON

Q292. Assertion (A): Adapter classes are always used instead of listener interfaces.
Reason (R): Adapter classes provide empty default implementations of every method in a multi-method listener interface.
Choose the correct option:

A. Both A and R are false

B. Both A and R are true, but R is NOT the correct explanation of A

C. Both A and R are true, and R is the correct explanation of A

D. A is false but R is true

Ans: D

Explanation: R is a true fact about adapters, but A is false — adapters are optional convenience classes, not a mandatory replacement for listener interfaces.

MATCH THE FOLLOWING

Q293. Match the AWT container/class in List-I with its default layout manager in List-II.
List-I: (1) Frame (2) Panel (3) Applet
List-II: (P) FlowLayout (Q) BorderLayout (R) FlowLayout

A. 1-Q, 2-P, 3-R
B. 1-Q, 2-R, 3-P
C. 1-R, 2-Q, 3-P
D. 1-P, 2-Q, 3-R

Ans: A

Explanation: Frame defaults to BorderLayout, while both Panel and Applet default to FlowLayout.

MATCH THE FOLLOWING

Q294. Match the applet life-cycle method in List-I with its purpose in List-II.
List-I: (1) init() (2) start() (3) destroy()
List-II: (P) One-time cleanup before removal (Q) One-time initial setup (R) Called each time applet becomes active

A. 1-R, 2-Q, 3-P
B. 1-P, 2-R, 3-Q
C. 1-Q, 2-R, 3-P
D. 1-Q, 2-P, 3-R

Ans: C

Explanation: init() sets up once, start() runs each time the applet becomes active, and destroy() cleans up once before removal.

COMPARISON

Q295. Which of the following correctly distinguishes Hashtable from Vector?

A. Hashtable stores an ordered list; Vector stores key-value pairs
B. Hashtable is unsynchronized; Vector is synchronized
C. Hashtable stores key-value pairs; Vector stores an ordered list of elements accessed by index
D. Both store identical key-value structures

Ans: C

Explanation: Hashtable is a map of keys to values, while Vector is an indexed, growable list of elements — both are synchronized.

COMPARISON

Q296. Which correctly contrasts String and StringBuffer?

A. Both are mutable

B. String is immutable; StringBuffer is mutable and supports in-place modification

C. String is mutable; StringBuffer is immutable

D. Neither can be concatenated

Ans: B

Explanation: String objects cannot be changed after creation, while StringBuffer allows characters to be appended, inserted or removed in place.

STATEMENT-BASED

Q297. Which statement about the CardLayout manager is INCORRECT?

A. It automatically arranges components in a grid of rows and columns

B. first() and last() can jump directly to the first or last card

C. Each component added is associated with a name/identifier

D. It displays only one component at a time

Ans: A

Explanation: Arranging components in a grid of rows and columns is the behavior of GridLayout, not CardLayout.

STATEMENT-BASED

Q298. Which statement about the Serializable interface is INCORRECT?

A. Implementing it enables an object to be converted to a byte stream

B. Fields marked transient are skipped during serialization

C. It is a marker interface with no methods

D. It forces the class to override writeObject() and readObject()

Ans: D

Explanation: Serializable itself declares no methods, so implementing it does not force writeObject()/readObject() to be overridden; that is optional.

CODE-BASED

Q299. What will the following code print?

```
StringBuffer sb = new StringBuffer("Java");
sb.append(" AWT");
System.out.println(sb);
```

A. JavaAWT
B. Java AWT
C. AWT Java
D. Java

Ans: B

Explanation: append() adds the given text to the end of the existing StringBuffer content, producing "Java AWT".

CODE-BASED

Q300. What is the output of the following code?

```
int a = 5, b = 2;
System.out.println(Math.pow(a, b));
```

A. 10.0
B. 32.0
C. 25.0
D. 7.0

Ans: C

Explanation: Math.pow(5, 2) computes 5 raised to the power 2, which equals 25.0 (returned as a double).

ASSERTION-REASON

Q301. Assertion (A): A Dialog box is commonly used for modal message boxes in AWT applications.
Reason (R): Dialog is a subclass of Window and is generally used with an owner Frame.
Choose the correct option:

A. Both A and R are true, and R is the correct explanation of A
B. A is false but R is true
C. Both A and R are true, but R is NOT the correct explanation of A
D. A is true but R is false

Ans: A

Explanation: Dialog does extend Window and is typically tied to an owning Frame, which is exactly why it suits modal message boxes.

MATCH THE FOLLOWING

Q302. Match the listener interface in List-I with the event it detects in List-II.
List-I: (1) AdjustmentListener (2) TextListener (3) ItemListener
List-II: (P) Checkbox/Choice selection change (Q) Scrollbar value change (R) Text content change

A. 1-R, 2-P, 3-Q
B. 1-Q, 2-R, 3-P
C. 1-Q, 2-P, 3-R
D. 1-P, 2-Q, 3-R

Ans: B

Explanation: AdjustmentListener detects Scrollbar value changes, TextListener detects text edits, and ItemListener detects selection changes.

COMPARISON

Q303. Which correctly distinguishes getCodeBase() from getDocumentBase() in an applet?

A. getCodeBase() returns the applet class's directory URL; getDocumentBase() returns the HTML page's directory URL
B. Neither method exists in the Applet class
C. getDocumentBase() returns the class file location; getCodeBase() returns the HTML page location
D. Both return identical URLs in every case

Ans: D

Explanation: getCodeBase() points to where the .class file was loaded from, while getDocumentBase() points to where the embedding HTML page resides; they can differ.

STATEMENT-BASED

Q304. Which statement about ScrollPane is INCORRECT?

A. It is intended to hold a single child component
B. It can hold multiple independently laid-out child components like Panel does
C. It automatically provides scrollbars when needed
D. It handles scrolling of its contained component automatically

Ans: C

Explanation: ScrollPane is designed to manage exactly one child component, unlike Panel, which can arrange multiple components via a layout manager.

CODE-BASED

Q305. What will the following code print?

```
Vector v = new Vector();
v.addElement("OAVS");
v.addElement("TGT");
System.out.println(v.size());
```

A. 3
B. 0
C. 2
D. 1

Ans: B

Explanation: Two elements were added to the Vector using addElement(), so size() correctly reports 2.

ASSERTION-REASON

Q306. Assertion (A): PopupMenu is typically shown at a position determined by a mouse event on some component, rather than being permanently fixed in a menu bar.
Reason (R): PopupMenu is designed to be displayed on demand rather than as a constant part of a MenuBar.
Choose the correct option:

A. Both A and R are true, but R is NOT the correct explanation of A
B. A is false but R is true
C. Both A and R are true, and R is the correct explanation of A
D. A is true but R is false

Ans: C

Explanation: PopupMenu's on-demand, position-based display (R) is exactly why it is triggered at a mouse-event location (A).

COMPARISON

Q307. Which correctly distinguishes the events generated by MenuItem and CheckboxMenuItem on selection?

A. MenuItem generates an ActionEvent; CheckboxMenuItem generates an ItemEvent reflecting its checked state
B. MenuItem generates ItemEvent while CheckboxMenuItem generates ActionEvent
C. Both generate only ActionEvent
D. Both generate only ItemEvent

Ans: A

Explanation: A plain MenuItem click fires ActionEvent, while a CheckboxMenuItem's toggle fires ItemEvent, similar to Checkbox.

STATEMENT-BASED

Q308. Which statement about the <applet> tag's 'codebase' attribute is INCORRECT?

A. It can point to a different directory or even a different host than the HTML page

B. It is optional, defaulting to the HTML document's own location if omitted

C. It specifies the pixel width of the applet's display area

D. It affects where the browser looks for the compiled .class file

Ans: C

Explanation: Specifying the pixel width is the job of the 'width' attribute, not 'codebase', which only controls where the class file is loaded from.

14 Mixed OAVS TGT Practice Set

MIXED PRACTICE

Q309. Which layout manager would you choose to build a simple calculator button pad of equal-sized buttons arranged in rows and columns?

A. CardLayout
B. GridLayout
C. FlowLayout
D. BorderLayout

Ans: B

Explanation: GridLayout arranges components into equal-sized cells, ideal for a uniform calculator button pad.

MIXED PRACTICE

Q310. Which method must be overridden to draw custom shapes directly on a Panel?

A. repaint(Component c)
B. paint(Graphics g)
C. init()
D. start()

Ans: B

Explanation: Overriding paint(Graphics g) lets a component perform custom rendering using the supplied Graphics object.

MIXED PRACTICE

Q311. A programmer wants a listener to handle only mouseClicked() without implementing the other four MouseListener methods. What should be used?

A. Extend MouseAdapter and override mouseClicked() only
B. Override paint() to detect clicks
C. Use an ActionListener instead
D. Implement MouseListener directly

Ans: A

Explanation: MouseAdapter supplies empty defaults for all five methods, so only mouseClicked() needs to be overridden.

MIXED PRACTICE

Q312. Which class would you use to read numeric and text lines from a text file efficiently, one line at a time?

A. ObjectInputStream
B. DataOutputStream
C. FileInputStream
D. BufferedReader wrapping a FileReader

Ans: D

Explanation: Wrapping a FileReader in a BufferedReader provides the readLine() method for efficient line-by-line text reading.

MIXED PRACTICE

Q313. Which class would you use to save (persist) an entire Java object, including its field values, to a file?

A. BufferedOutputStream
B. ObjectOutputStream with writeObject()
C. FileWriter with toString()
D. PrintStream

Ans: B

Explanation: ObjectOutputStream.writeObject() serializes and writes the complete state of an object to a stream.

MIXED PRACTICE

Q314. In an applet, where should you place code that must run exactly once when the applet is first loaded, such as loading an image?

A. stop()
B. init()
C. start()
D. paint()

Ans: B

Explanation: init() is invoked only once during the applet's lifetime, making it the right place for one-time setup work.

MIXED PRACTICE

Q315. Which statement correctly differentiates FlowLayout from BorderLayout?

A. FlowLayout has 5 fixed regions; BorderLayout flows components left to right

B. BorderLayout has 5 fixed regions (N,S,E,W,CENTER); FlowLayout flows components left to right, wrapping lines

C. Both arrange components identically

D. BorderLayout is the default for Panel

Ans: B

Explanation: BorderLayout uses five named regions, while FlowLayout simply lines components up left to right, wrapping as needed.

MIXED PRACTICE

Q316. Which method should be called on a Vector to retrieve the element stored at a specific index?

A. valueAt(int index)

B. elementAt(int index)

C. get(index) only

D. fetch(int index)

Ans: B

Explanation: elementAt(int index) is the classic Vector method that returns the element at the given index.

MIXED PRACTICE

Q317. Which utility class method would you use to generate a random integer between 0 (inclusive) and a bound (exclusive)?

A. System.random(bound)

B. Vector.random(bound)

C. Math.random(bound)

D. new Random().nextInt(bound)

Ans: D

Explanation: Random.nextInt(int bound) returns a pseudo-random int between 0 (inclusive) and bound (exclusive).

MIXED PRACTICE

Q318. Which AppletContext method allows one applet to obtain a reference to another applet named "clock" on the same page?

A. getAppletInfo()
B. getApplets()
C. showDocument("clock")
D. getApplet("clock")

Ans: D

Explanation: getApplet(String name) looks up and returns another applet on the page by its declared name, enabling communication.

MIXED PRACTICE

Q319. Why does the File class NOT provide methods to read or write the actual content of a file?

A. Because File extends InputStream
B. Because File is deprecated
C. Because File can only represent directories
D. Because File only represents a pathname/ metadata; actual data transfer is handled by stream classes

Ans: D

Explanation: File models an abstract file/directory path and its properties; reading/writing bytes or characters is delegated to stream and reader/writer classes.

MIXED PRACTICE

Q320. Which keyword, when applied to a field, ensures that field's value is NOT saved when its containing object is serialized?

A. private
B. static
C. final
D. transient

Ans: D

Explanation: The transient keyword explicitly excludes a field from the serialized byte stream of an object.

MIXED PRACTICE

Q321. A form needs a component where the user types free-form multi-line feedback. Which AWT component is most appropriate?

A. Label
B. Choice
C. TextArea
D. TextField

Ans: D

Explanation: TextArea supports multiple lines of editable text, making it suitable for longer free-form input like feedback.

MIXED PRACTICE

Q322. Which method would you call on a Frame after adding all its components to make the window automatically size itself to fit them?

A. pack()
B. setSize()
C. setVisible()
D. validate()

Ans: B

Explanation: pack() sizes the Frame based on the preferred sizes of its child components rather than a manually specified size.

MIXED PRACTICE

Q323. An applet needs to load and later play a short sound effect. Which two AppletContext-related calls would it typically use, in order?

A. getAudioClip() then play()
B. showDocument() then play()
C. getApplet() then run()
D. getImage() then drawImage()

Ans: D

Explanation: The applet first fetches an AudioClip using getAudioClip(URL), then invokes play() on that clip to play the sound.

MIXED PRACTICE

Q324. Which utility class method would best generate a unique-ish timestamp to name a temporary log entry?

A. Vector.size()
B. Math.random()
C. Random.nextInt()
D. System.currentTimeMillis()

Ans: B

Explanation: System.currentTimeMillis() returns the current time, commonly used to generate simple time-based identifiers.

MIXED PRACTICE

Q325. Which File class method would you check before calling delete(), to confirm a directory has no files left inside it (relevant since File.delete() only removes empty directories)?

A. exists() == false

B. isEmpty()

C. list().length == 0

D. size() == 0

Ans: C

Explanation: Since delete() fails on a non-empty directory, checking that list() returns an empty array confirms it is safe to delete.

MIXED PRACTICE

Q326. An application needs a top menu bar with a 'File' menu containing 'Open' and 'Save' options. Which combination of AWT classes is appropriate?

A. A List with two entries "Open" and "Save"

B. A MenuBar containing a Menu named "File", with MenuItem entries "Open" and "Save"

C. A Choice containing items "Open" and "Save"

D. A PopupMenu attached directly to the Frame only

Ans: B

Explanation: MenuBar holds Menu objects, each of which holds MenuItem entries -- the standard way to build a File menu.

MIXED PRACTICE

Q327. An application needs to let the user browse and pick a file using the operating system's native file picker. Which class is most appropriate?

A. File.list() displayed in a Label

B. FileDialog

C. TextField with manual path entry only

D. Choice populated with filenames

Ans: B

Explanation: FileDialog opens the native OS file-selection dialog box for choosing a file to open or save.

MIXED PRACTICE

Q328. Which call sequence would correctly play a looping background sound clip in an applet?

A. AudioClip clip = getImage(url); clip.play();

B. getAppletContext().getImage(url).loop();

C. AudioClip clip = getAppletContext().getAudioClip(url); clip.loop();

D. getAppletContext().play(url);

Ans: C

Explanation: The clip must first be obtained via getAudioClip(URL), then played repeatedly using its own loop() method.

MIXED PRACTICE

Q329. To compare two String variables for equal CONTENT (not reference) inside an if-condition, which is correct?

A. if (str1.same(str2))

B. if (str1 == str2)

C. if (str1.compareTo(str2))

D. if (str1.equals(str2))

Ans: D

Explanation: equals() checks character-by-character content equality; == merely checks whether both variables reference the same object.